

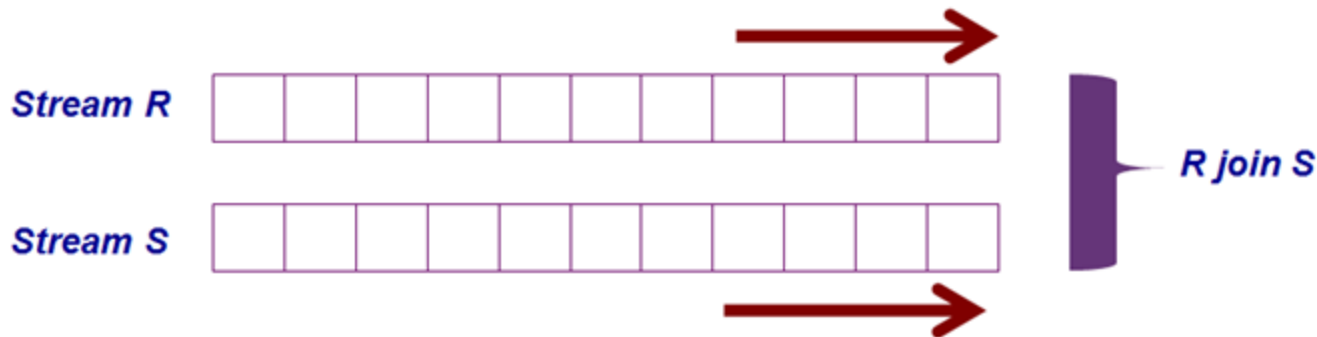
FIT5202 – Data Processing for Big Data



- Streaming Data Processing
- Streaming Processing Technology

- Overview of Stream join
- Time based window stream join (Unbounded)
 - Tuple slide
 - Time slide
- Tuple based window stream join (Unbounded)
- Bounded stream join

Overview of Stream Join



Bounded Stream Join

- If both streams R and S are bounded streams
- There is a start and an end

Unbounded Stream Join

- If any of the streams is unbounded
- There is a start but no end

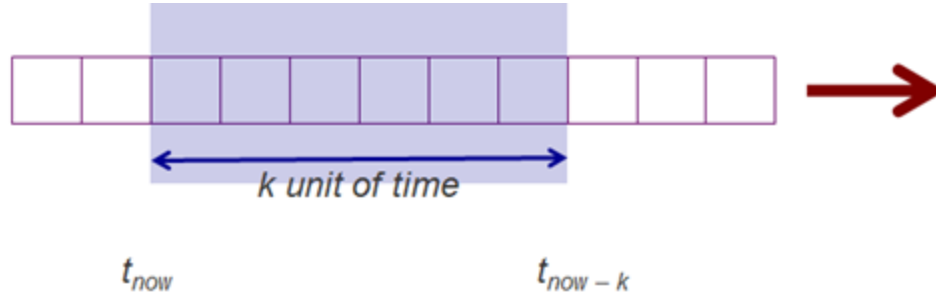
Challenges:

- ❑ How to **join streams with no end**?
- ❑ Due to network latency, **some data arrives late** compared to other related tuples
 - How to join earlier and later tuples?

Recap - Windowing Mechanism in Unbounded Streams

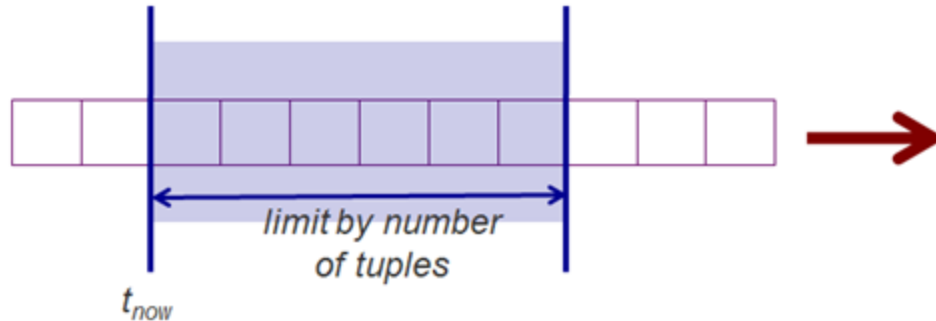
Time-based Window

Window size based on fixed unit of time

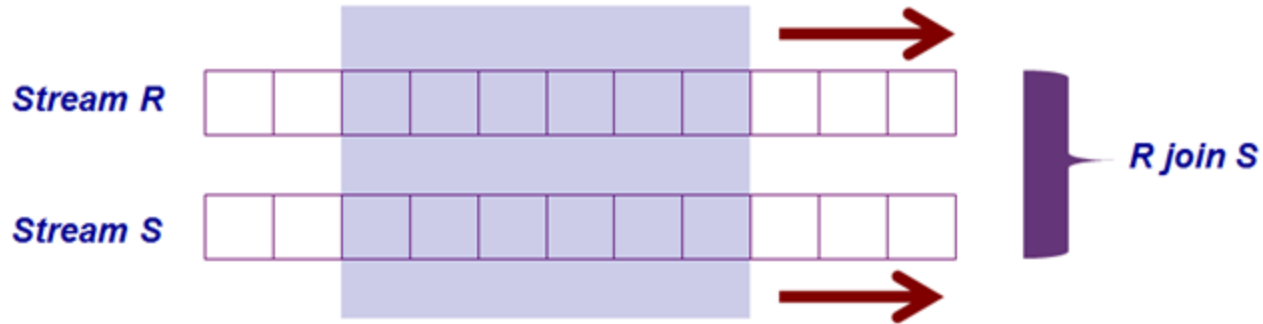


Tuple-based Window (Count-based Window)

Window size based on fixed number of tuples in window



Window-based Stream Join



A General Stream Join Process:

- When a tuple r arrives from input stream R :
 - Scan stream S 's window to find tuples matching r , and get join result
 - Insert new tuple r into window for stream R
 - Invalidate all expired tuples in stream R 's window

Hence, stream join is based on the data in the current window!!

- Tuples in the window cannot be matched with expired tuples (discarded from window) and with incoming tuples (not arrived yet)

Review – Hash Join

Hash S Table

Table S	
Arts	8
Business	15
CompSc	2
Dance	12
Engineering	7
Finance	21
Geology	10
Health	11
IT	16

hashed
into
→

Hash Table	
Index	Entries
1	Geology/10
2	CompSc/2
3	Dance/12
4	
5	
6	Business/15
7	Engineering/7
8	Arts/8
9	IT/18
10	
11	
12	

Health/11
Finance/21

Figure 5.6. Hashing Table S

Probe Table R into hash table

Table R	
Adele	8
Bob	22
Clement	16
Dave	23
Ed	11
Fung	25
Goel	3
Harry	17
Irene	14
Joanna	2
Kelly	6
Lim	20
Meng	1
Noor	5
Omar	19

probed
into

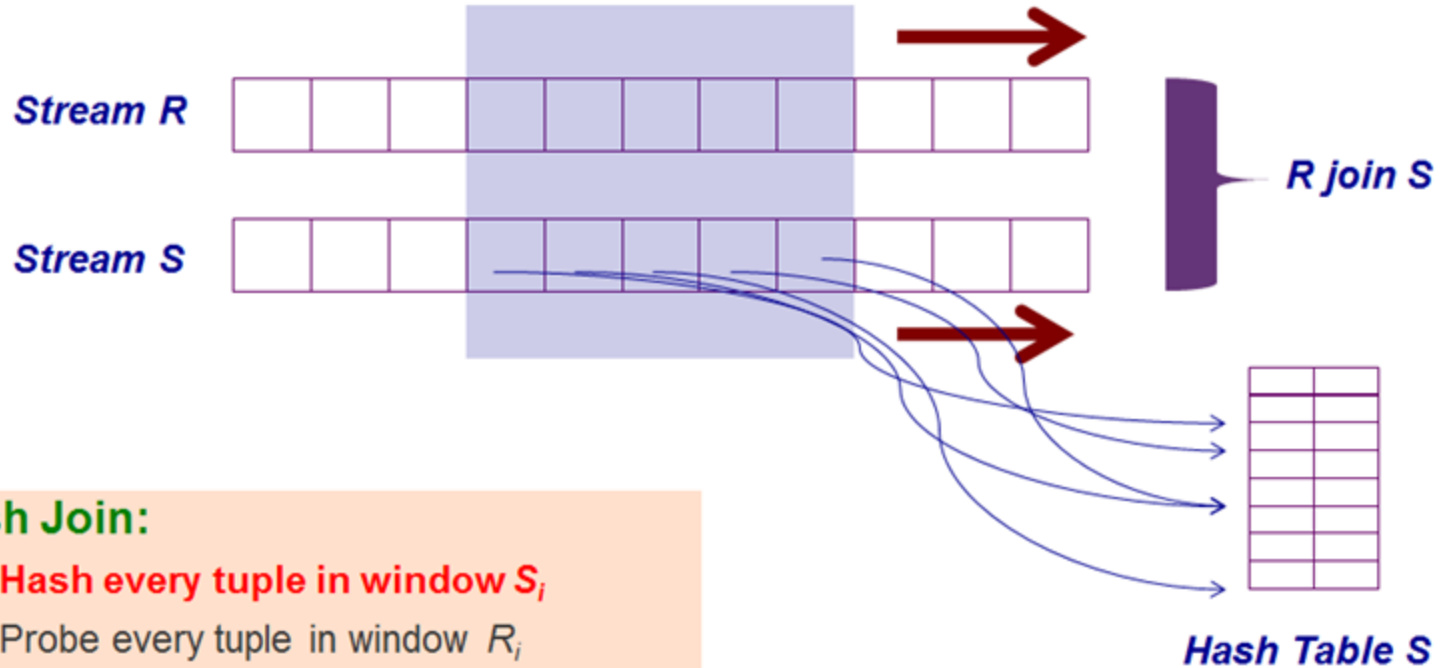
Hash Table	
Index	Entries
1	Geology/10
2	CompSc/2
3	Dance/12
4	
5	
6	Business/15
7	Engineering/7
8	Arts/8
9	IT/18
10	
11	
12	

Health/11
Finance/21

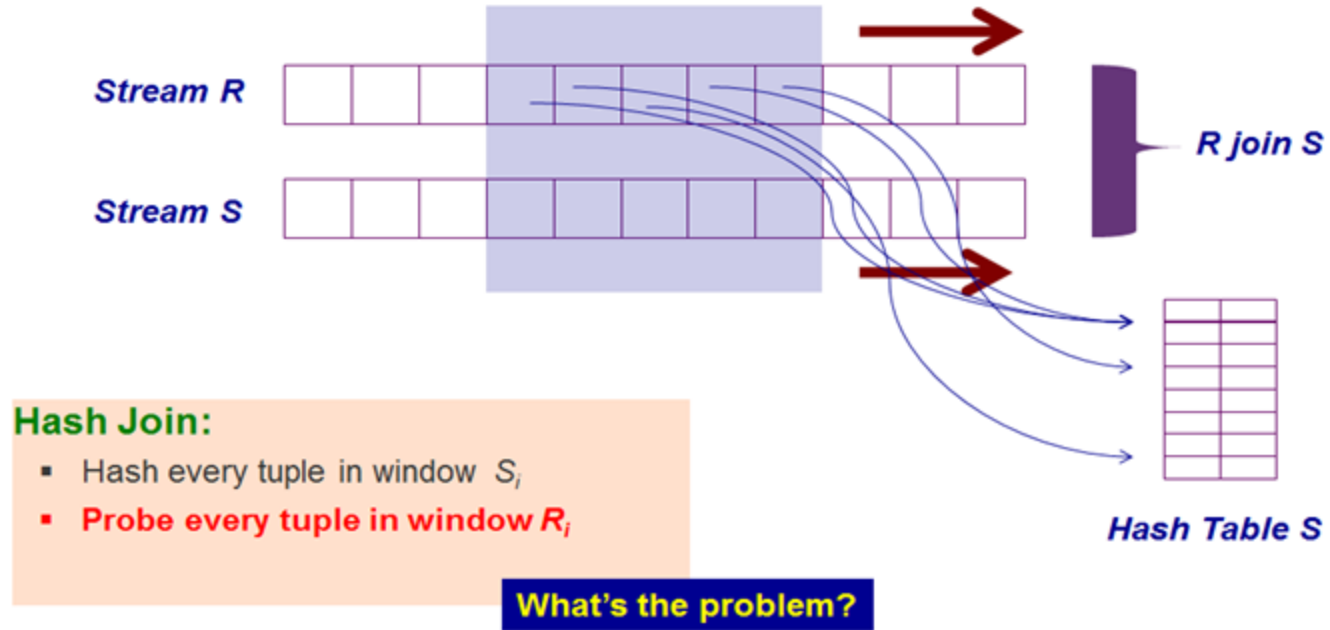
Join Results		
Adele	8	Arts
Ed	11	Health
Joanna	2	CompSc

Figure 5.7. Probing Table R

Hash Join



Hash Join



Problem: If r comes in first and then s comes in later, then r will not be compared with the s
→ If r and s has same key, we will miss join operation

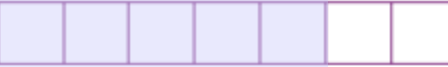
Symmetric Hash Join

Step 1:

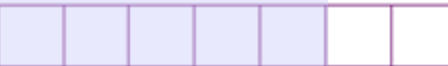
tuple r arrives

r

Stream R



Stream S



tuple s not yet
processed

s



Hash Table R



$R \text{ join } S$



Hash Table S

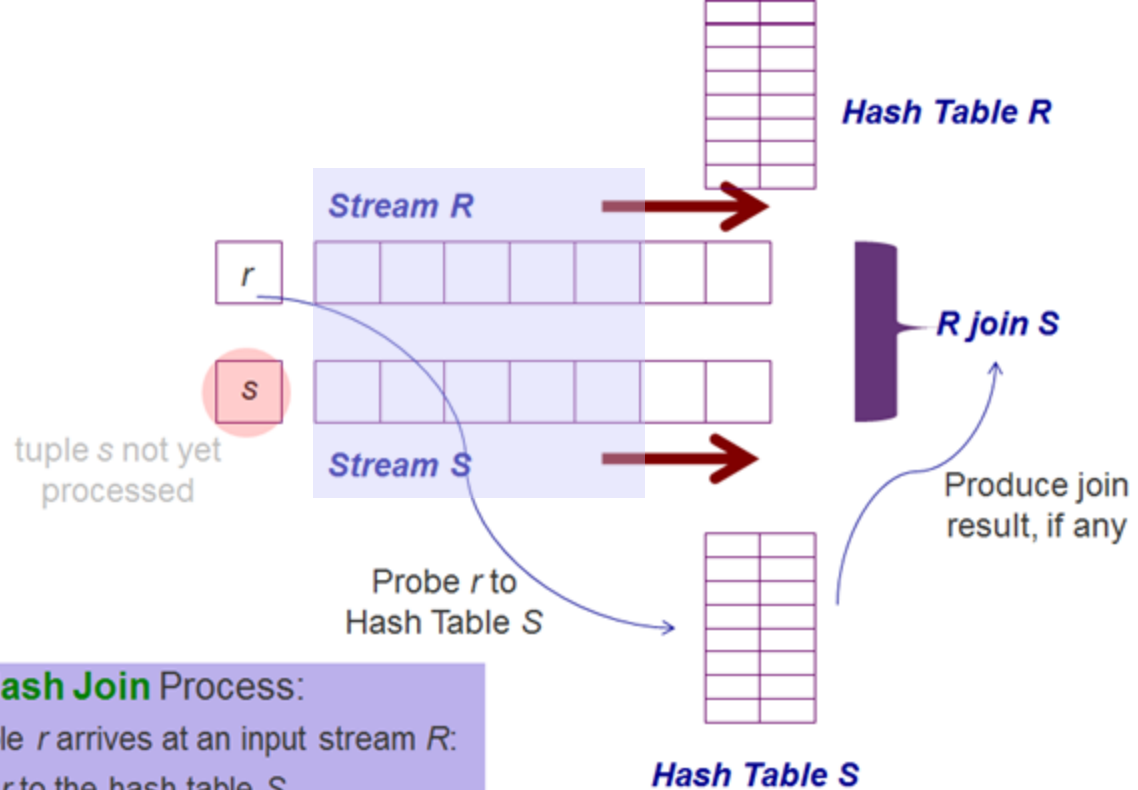
Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

Symmetric Hash Join

Step 2:

Probe r into hash table S



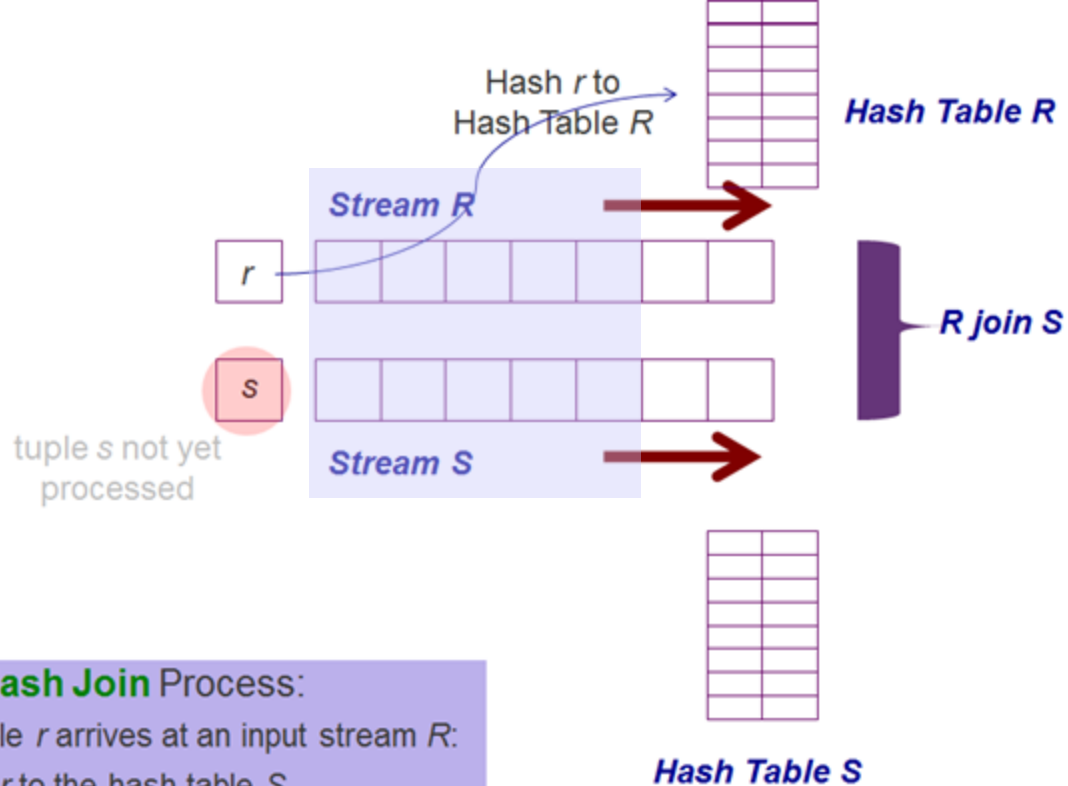
Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

Symmetric Hash Join

Step 3:

Hash r into hash table R



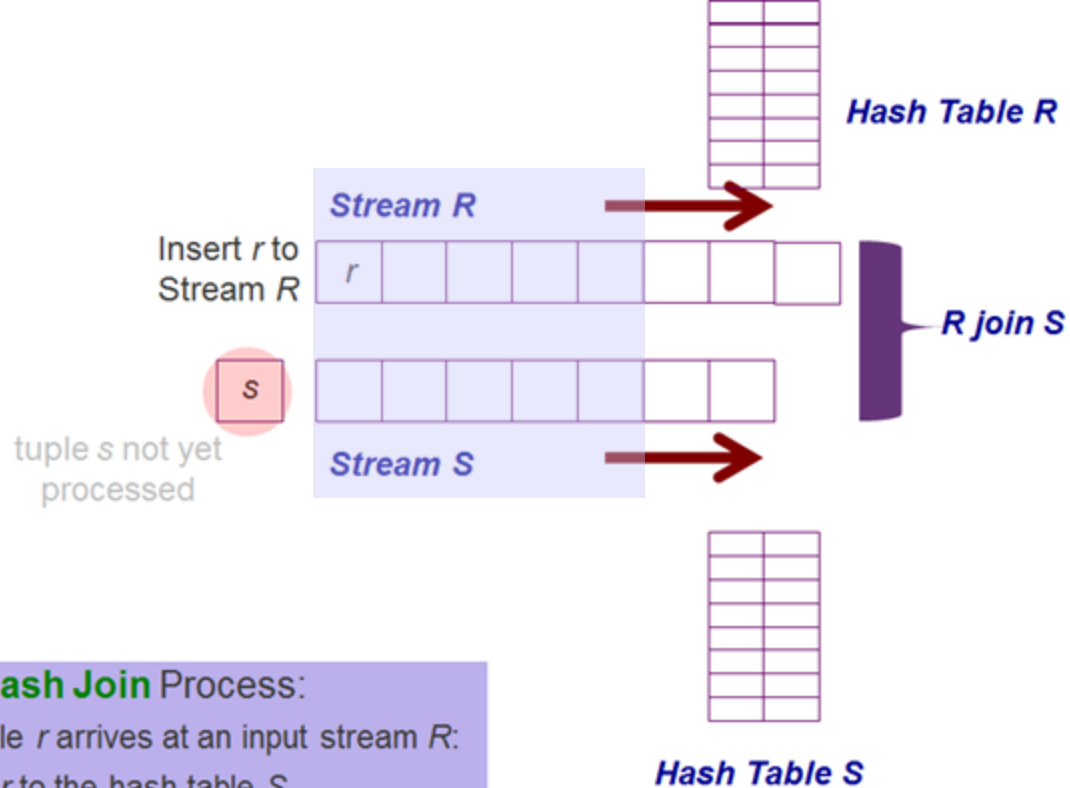
Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

Symmetric Hash Join

Step 4:

- When s comes in, probe into hash table R to get join results
- After that, hash s into own hash table, and insert into stream S



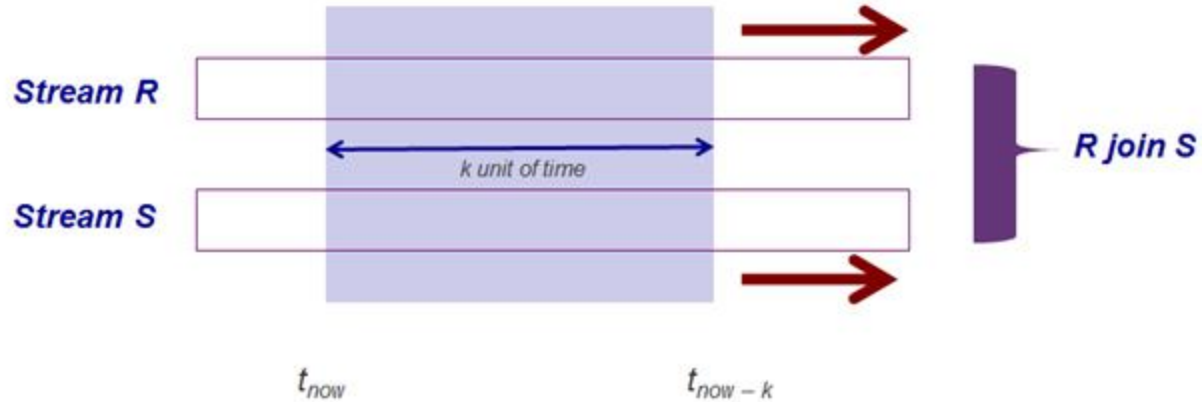
Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

By using two hash tables (symmetric), we don't miss the join of any incoming tuple.

- Overview of Stream join
- Time based window stream join (Unbounded)
 - Tuple slide
 - Time slide
- Tuple based window stream join (Unbounded)
- Bounded stream join

Time-based Window Stream Join



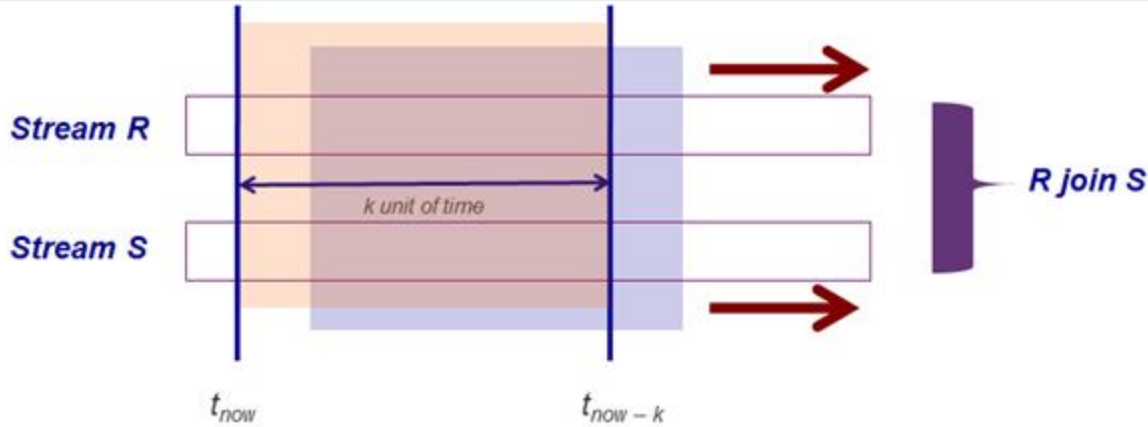
Unbounded Stream Join (Window-based):

- Join is only applied to tuples in the window
- But window is a running window...

Each window has fixed time duration

- Any numbers of tuples within that window will be considered for the join operation

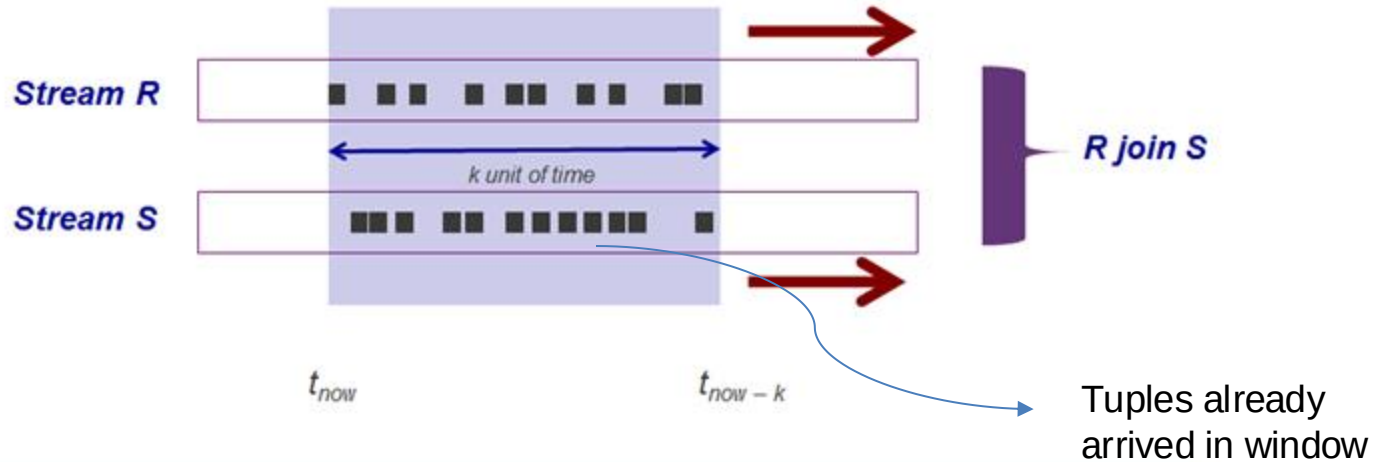
Time-based Window Stream Join



Unbounded Stream Join (Window-based):

- Join is only applied to tuples in the window
- **But window is a running window...**
- **How to slide the window?**
 - **Tuple Slide** - Slide window based on number of tuples (move window when tuples come in)
 - **Time Slide** - Slide window based on time interval (e.g., move window every 30s)

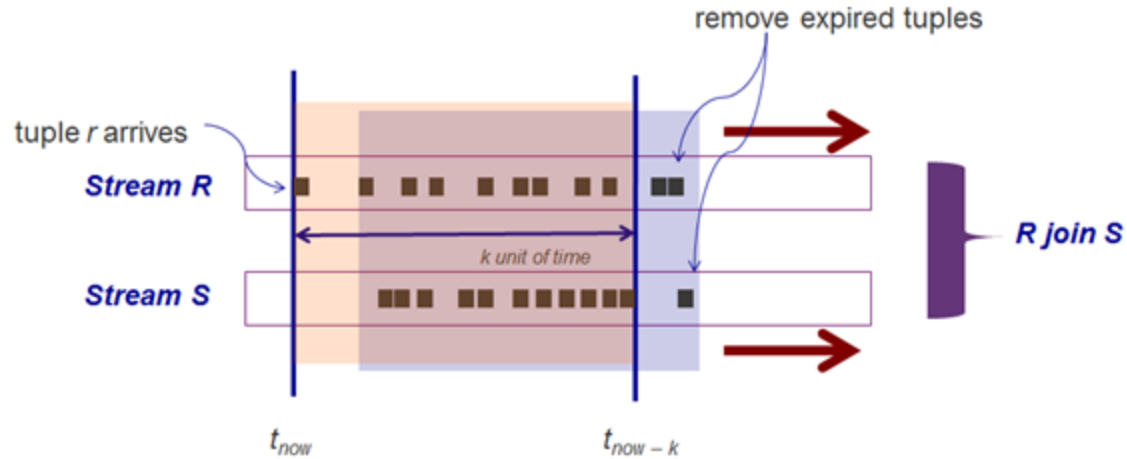
Tuple Slide Stream Join



Tuple Slide Stream Join:

- When a new tuple r arrives at stream R
 - Slide all the windows (remove expired tuples)
 - Join r with all tuples in the new window in stream S
 - Add r to the window in stream R

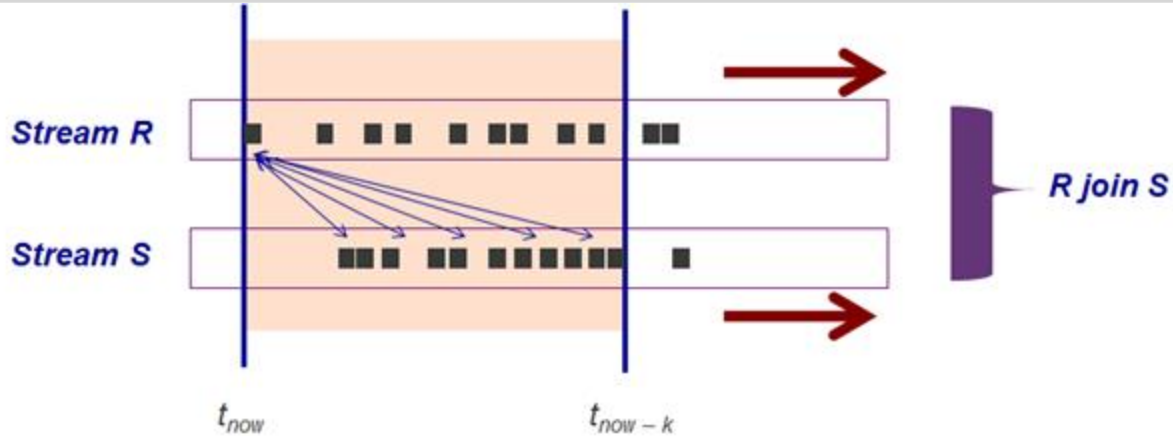
Tuple Slide Stream Join



Tuple Slide Stream Join:

- When a new tuple r arrives at stream R
 - **Slide all the windows (remove expired tuples)**
 - Join r with all tuples in the new window in stream S
 - Add r to the window in stream R

Tuple Slide Stream Join



Tuple Slide Stream Join:

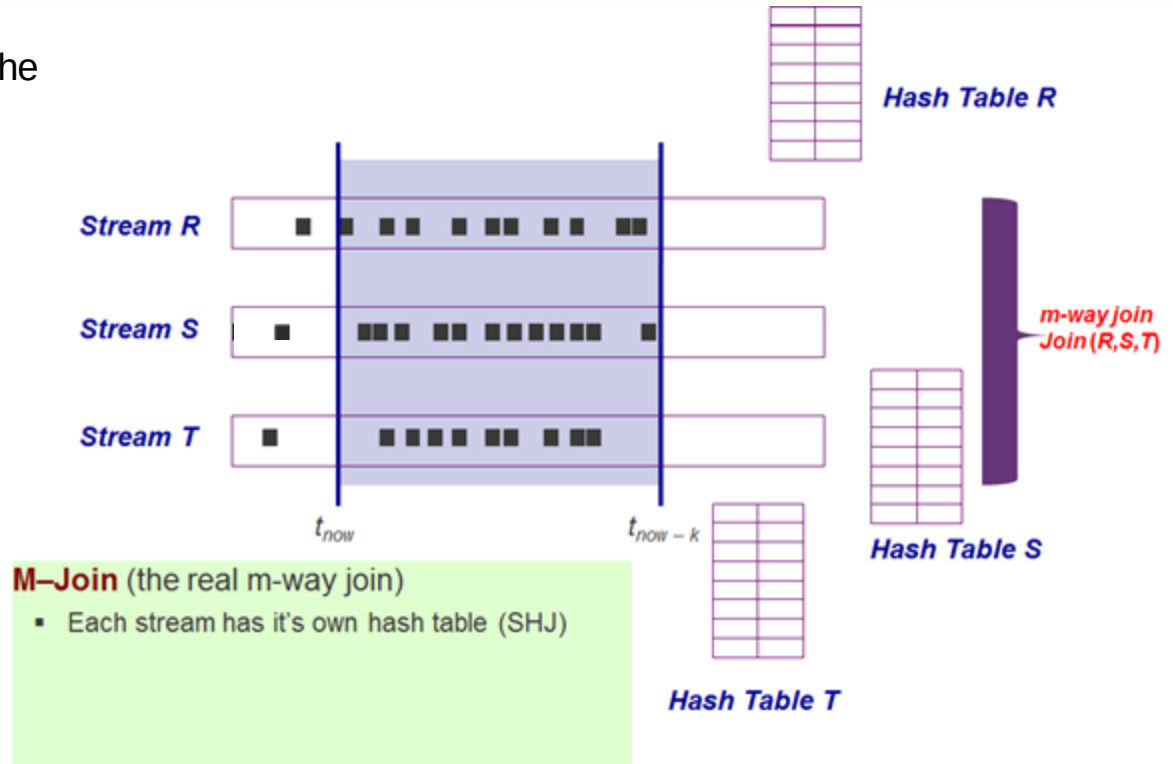
- When a new tuple r arrives at stream R
 - Slide all the windows (remove expired tuples)
 - Join r with all tuples in the new window in stream $S \rightarrow$ which join method?
 - Add r to the window in stream R

→ Symmetric Hash join

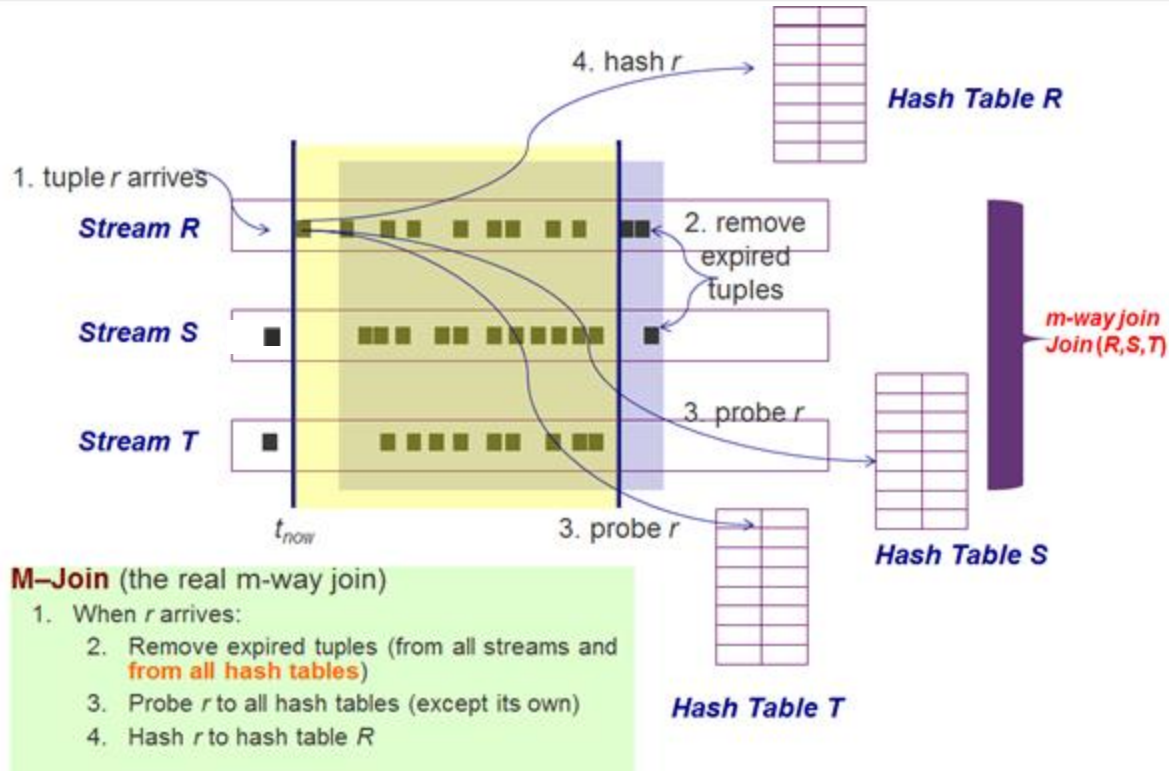
Tuple Slide (Using M-Join)

How to join more than two streams at the same time?

- ❑ Use **M-Join** – a multiway streaming join
- ❑ Based on symmetric hash join (work similarly to two-stream)



Tuple Slide (Using M-Join)



Repeat same procedure when tuple s arrives

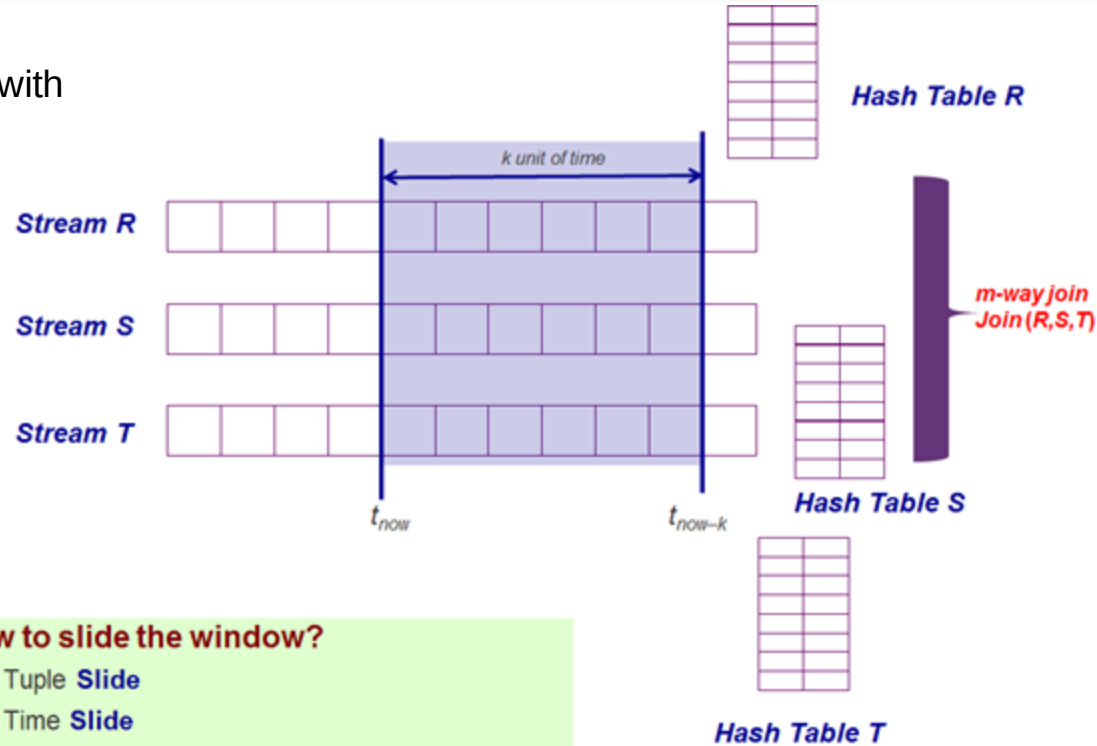
- ❑ Probe s into hash table R & T, and find the match
- ❑ Then, hash s to hash table S

Advantages: can still match r and s although s arrive later than r

Repeat the same when tuple t arrives

Time Slide (Using M-Join)

How M-join works with time-slide window

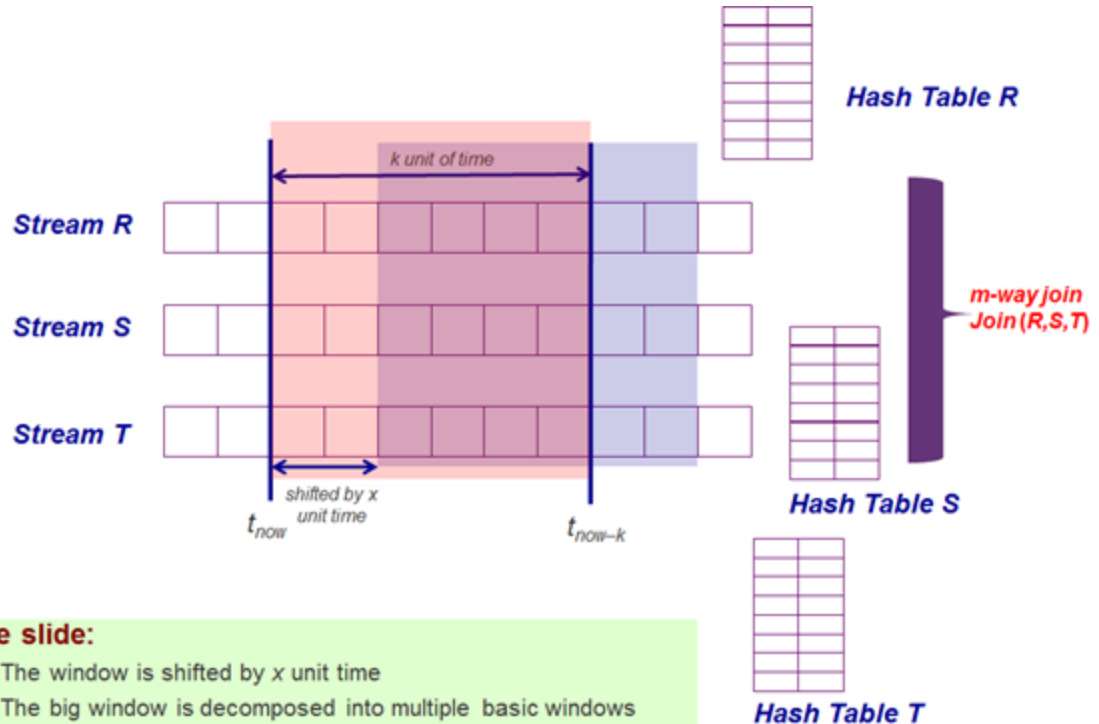


Time Slide (Using M-Join)

- Let one box here represents one **unit time** or one **basic window**.

Ex: 1 unit time = 1 minute

- Window size = 6 mins
- slide = 2mins



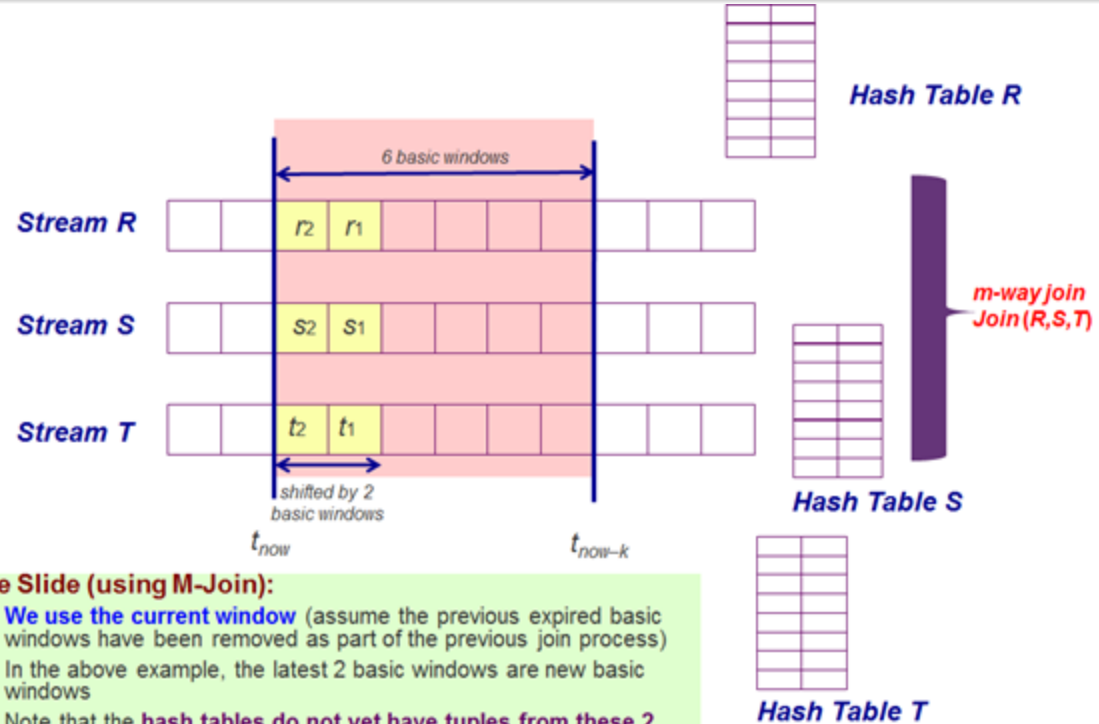
Time slide:

- The window is shifted by x unit time
- The big window is decomposed into multiple basic windows
- The big window is then **shifted by 1 or more basic windows**

Time Slide (Using M-Join)

Question:

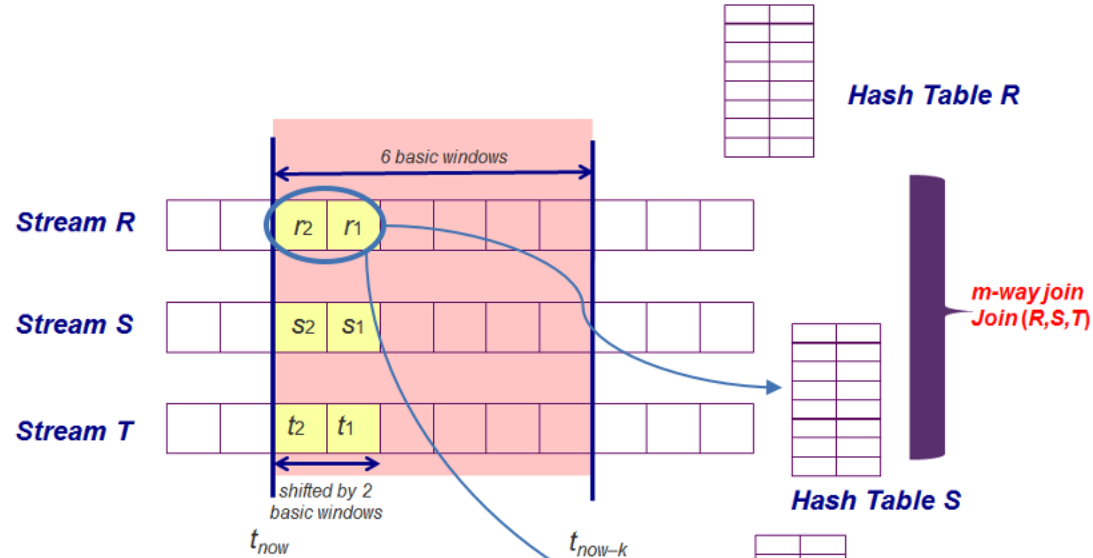
How to process or join tuples in the new basic windows?



Time Slide (using M-Join):

- We use the **current window** (assume the previous expired basic windows have been removed as part of the previous join process)
- In the above example, the latest 2 basic windows are new basic windows
- Note that the **hash tables do not yet have tuples from these 2 basic windows**

Time Slide (Using M-Join)

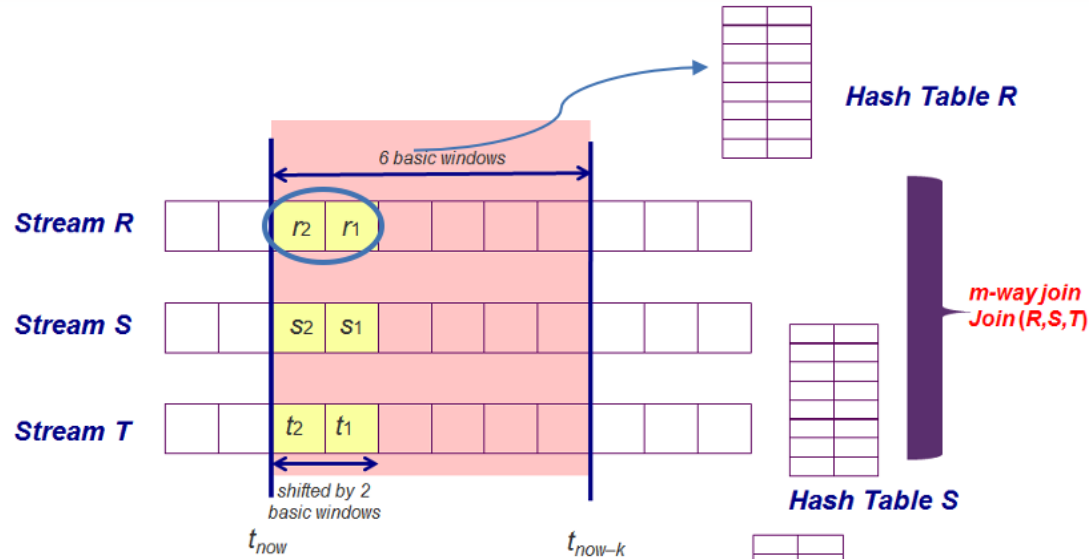


Time Slide (using M-Join):

▪ Process Stream R:

- Take all tuples from basic windows $r1$ and $r2$, and probe to hash tables S and T
- Take all tuples from basic windows $r1$ and $r2$ to hash table R

Time Slide (Using M-Join)

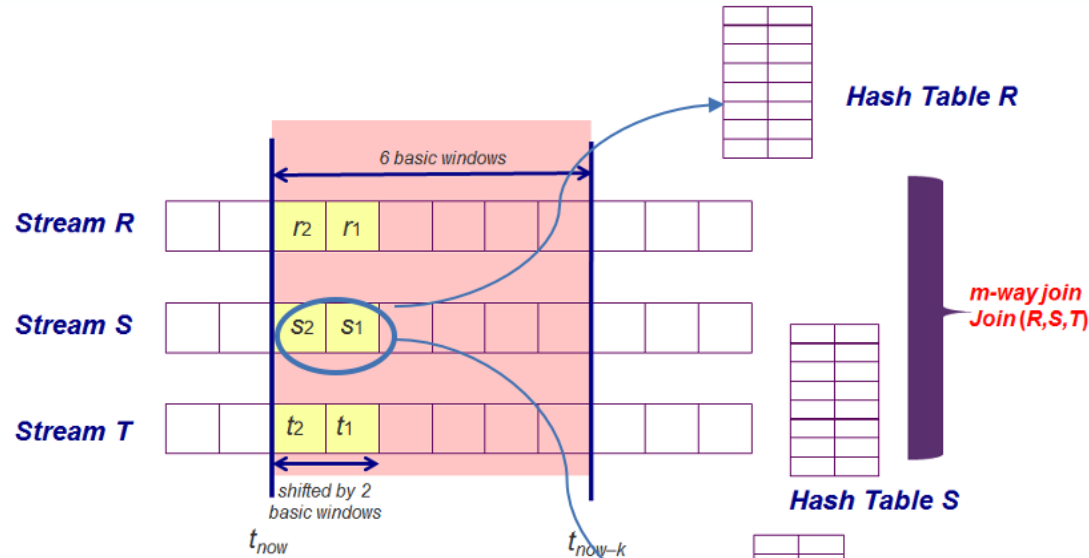


Time Slide (using M-Join):

▪ Process Stream R:

- Take all tuples from basic windows r_1 and r_2 , and probe to hash tables S and T
- Take all tuples from basic windows r_1 and r_2 to hash table R

Time Slide (Using M-Join)

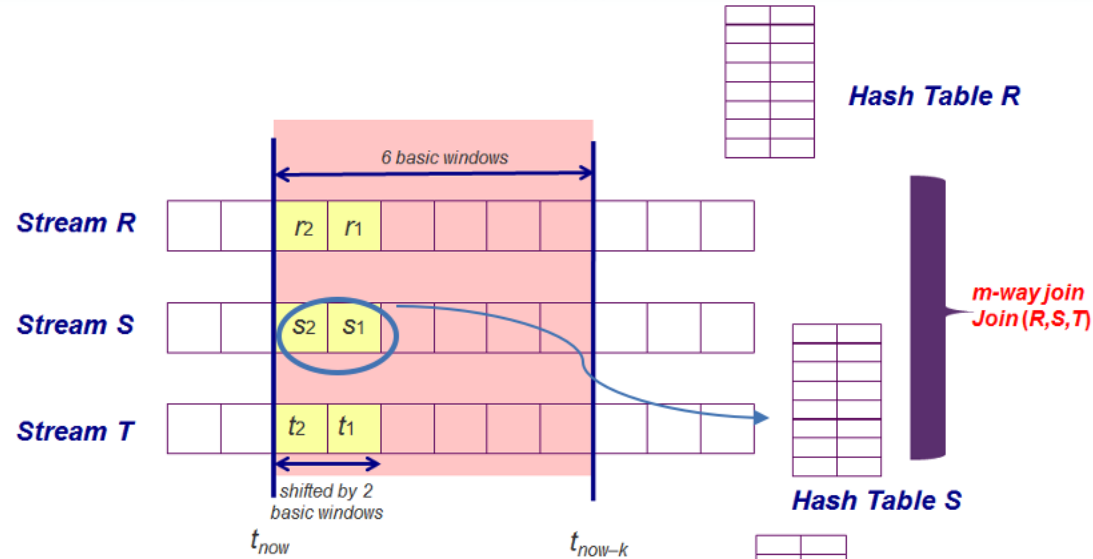


Time Slide (using M-Join):

▪ Process Stream S:

- Take all tuples from basic windows $s1$ and $s2$, and probe to hash tables R and T
- Take all tuples from basic windows $s1$ and $s2$ to hash table S

Time Slide (Using M-Join)

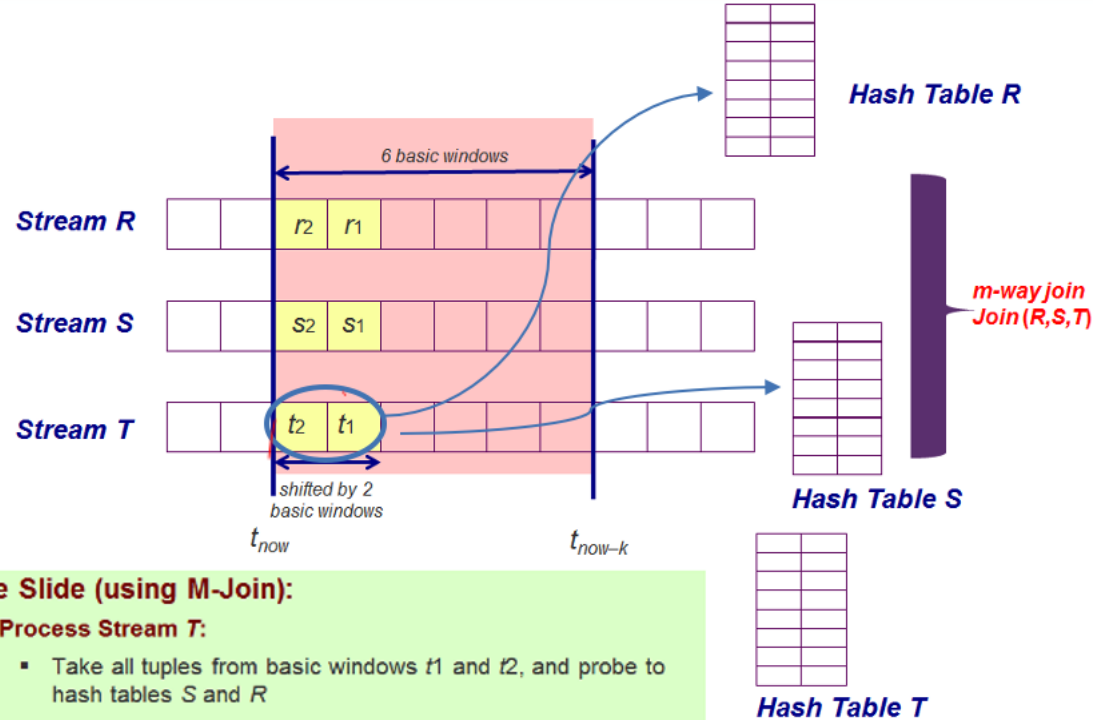


Time Slide (using M-Join):

▪ Process Stream S:

- Take all tuples from basic windows s1 and s2, and probe to hash tables R and T
- Take all tuples from basic windows s1 and s2 to hash table S

Time Slide (Using M-Join)

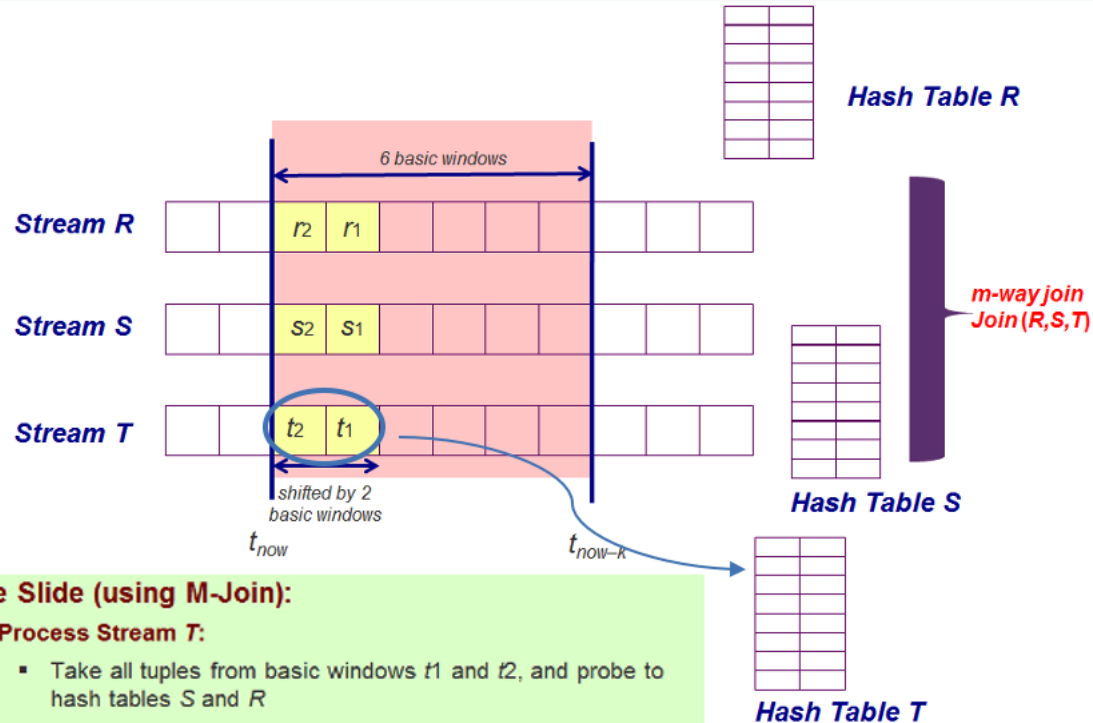


Time Slide (using M-Join):

▪ Process Stream T:

- Take all tuples from basic windows t_1 and t_2 , and probe to hash tables S and R
- Take all tuples from basic windows t_1 and t_2 to hash table T

Time Slide (Using M-Join)



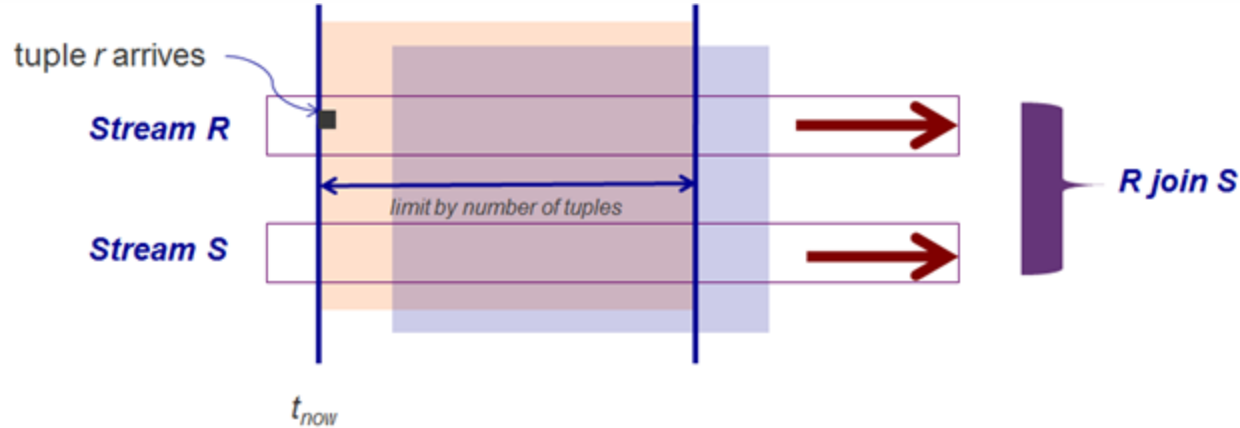
Time Slide (using M-Join):

▪ Process Stream T:

- Take all tuples from basic windows t_1 and t_2 , and probe to hash tables S and R
- Take all tuples from basic windows t_1 and t_2 to hash table T

- Overview of Stream join
- Time based window stream join (Unbounded)
 - Tuple slide
 - Time slide
- Tuple based window stream join (Unbounded)
- Bounded stream join

Tuple-based Window Stream Join

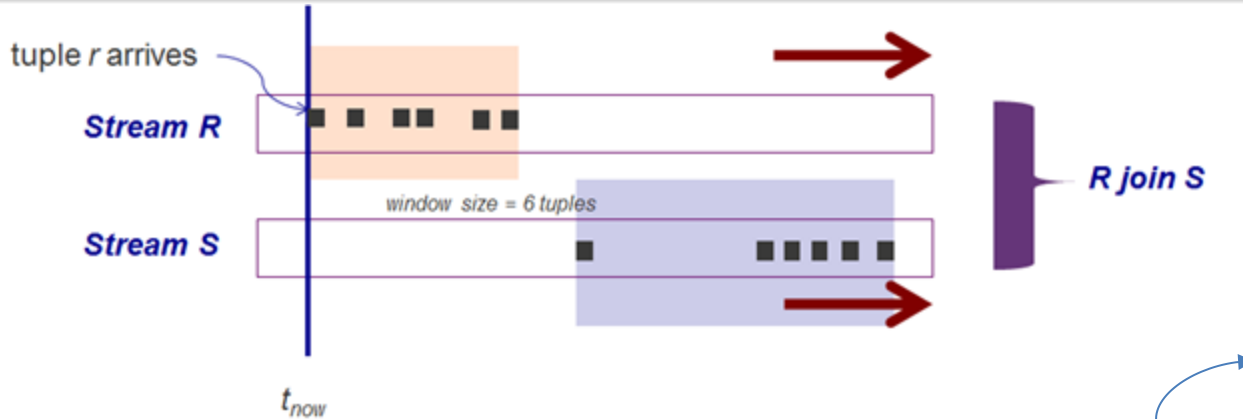


Ex: window size =100 tuples

- When 1 tuple comes in , oldest tuple needs to be removed to include it in the window because the size has to be 100

- **Size of the window** is k number of tuples
- When a tuple r arrives in stream R , we readjust the window size of stream R . What about stream S ? If the size of the window is based on stream R , window in stream S might have less number of tuples (thus violates the tuple-based window rule)
- If we allow **window size to be different**, what is the **semantic of the window**?

Tuple-based Window Stream Join



- Should we have the **same window size** for all streams in the join?
- If this is the case, in an extreme case, windows among streams will not overlap
- What is the semantic of the window, and hence the join?

Not many research in Tuple-based Window Join, because lack of understanding on how tuple-based window may be applied to stream join.

- ❑ If we slide windows for both R & S at the same time → **window size will be different**
- ❑ OR, we have different sliding mechanism for S, e.g., no slide until it has new tuples → **same window size**
 - **Problem:** If two streams have different arrival rate (e.g. R is faster than S)
 - Tuples in S will get very old
- ❑ When performing join, you join new data in R with old data in S

Inspired by how soccer players handshake

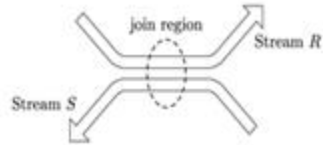


Figure 1: Handshake join idea.

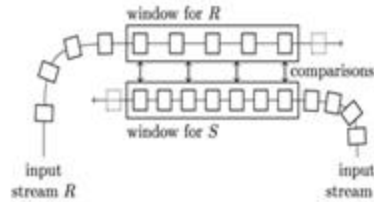


Figure 3: Handshake join sketch. Streams flow by each other in opposite directions; comparisons (and result generation) happens in parallel as the streams pass by.

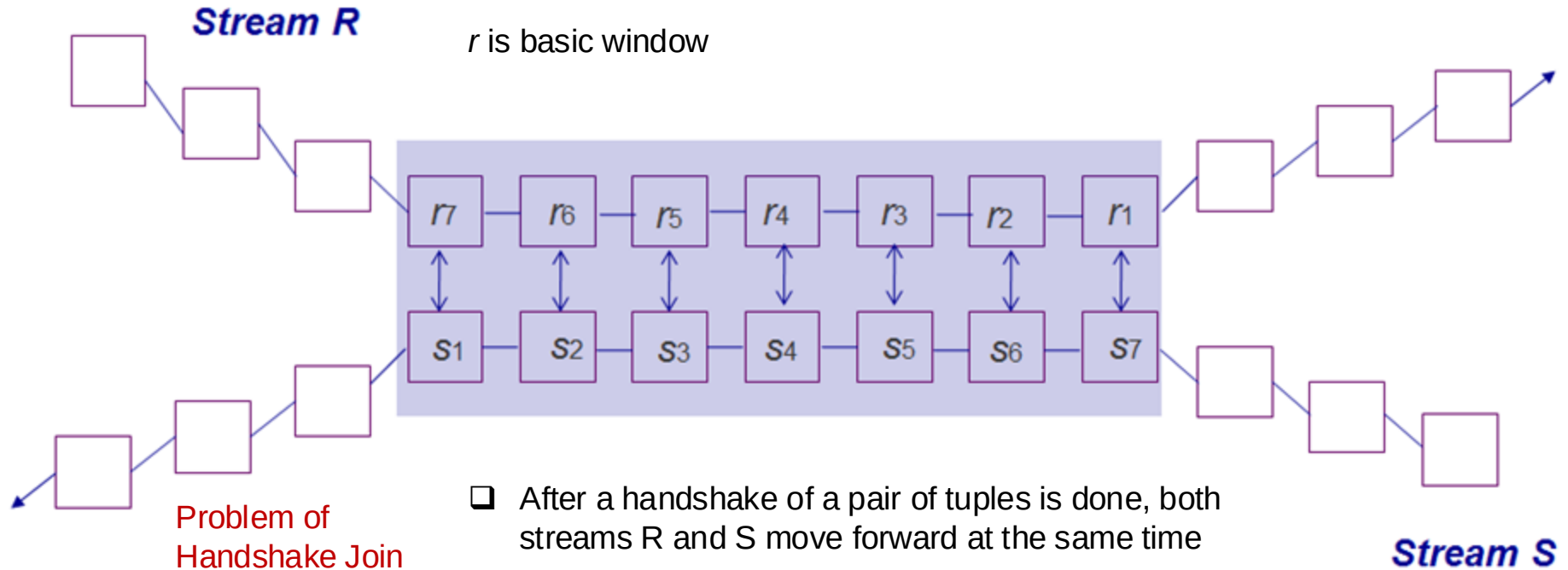
A strategy proposed to join very new tuples in one stream with old tuples in another stream

- Even then... this is not a Tuple-based Window Stream Join
- It is still a Time-based Window Stream Join
- But monitoring each tuple (and when it is expired) looks more natural in this Soccer Handshake method.

- ❑ Streams flow by each other in opposite direction.
- ❑ Ex: stream R moves from left to right and stream S moves from right to left
- ❑ Comparison/join occurs when streams pass by

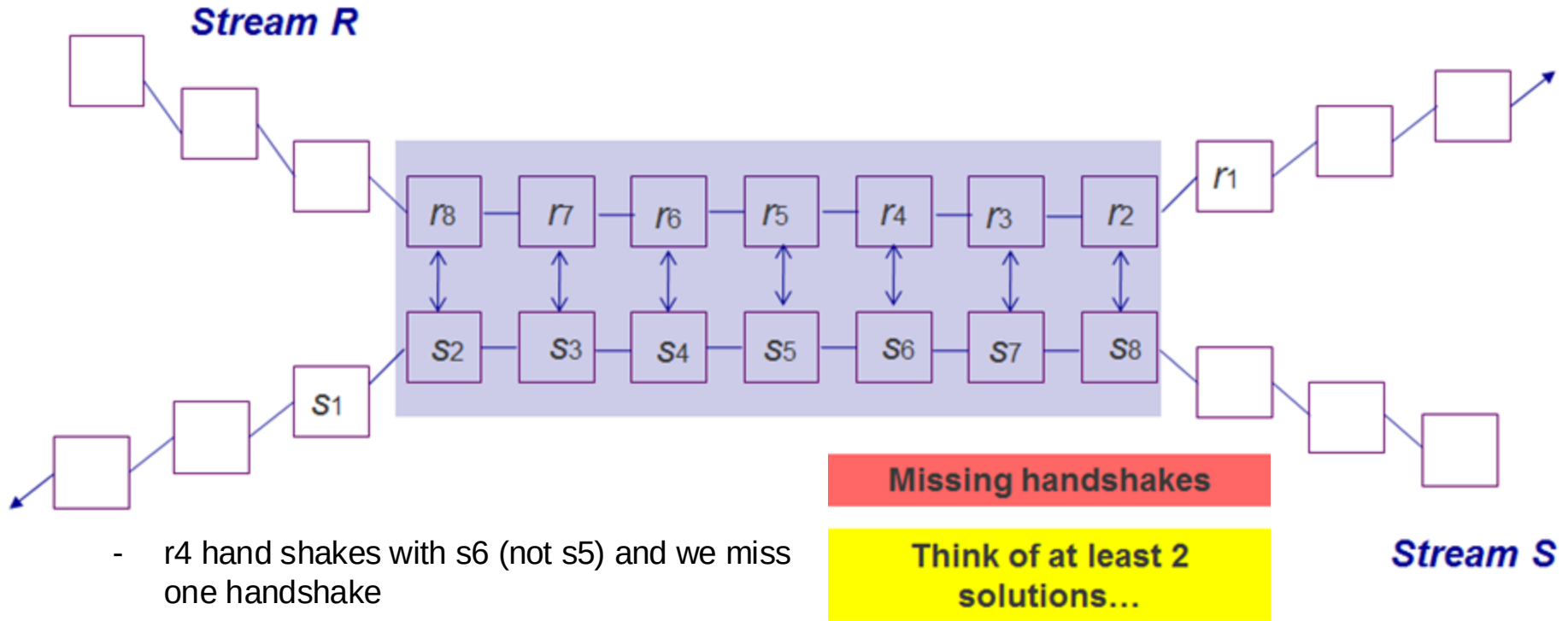
Looks nice, but there are problems...

Handshake Join



- ❑ After a handshake of a pair of tuples is done, both streams R and S move forward at the same time
- ❑ As a result, we will miss a handshake because one tuple from R moves to the right, and one tuple from S moves to the left, and one handshake will be missed

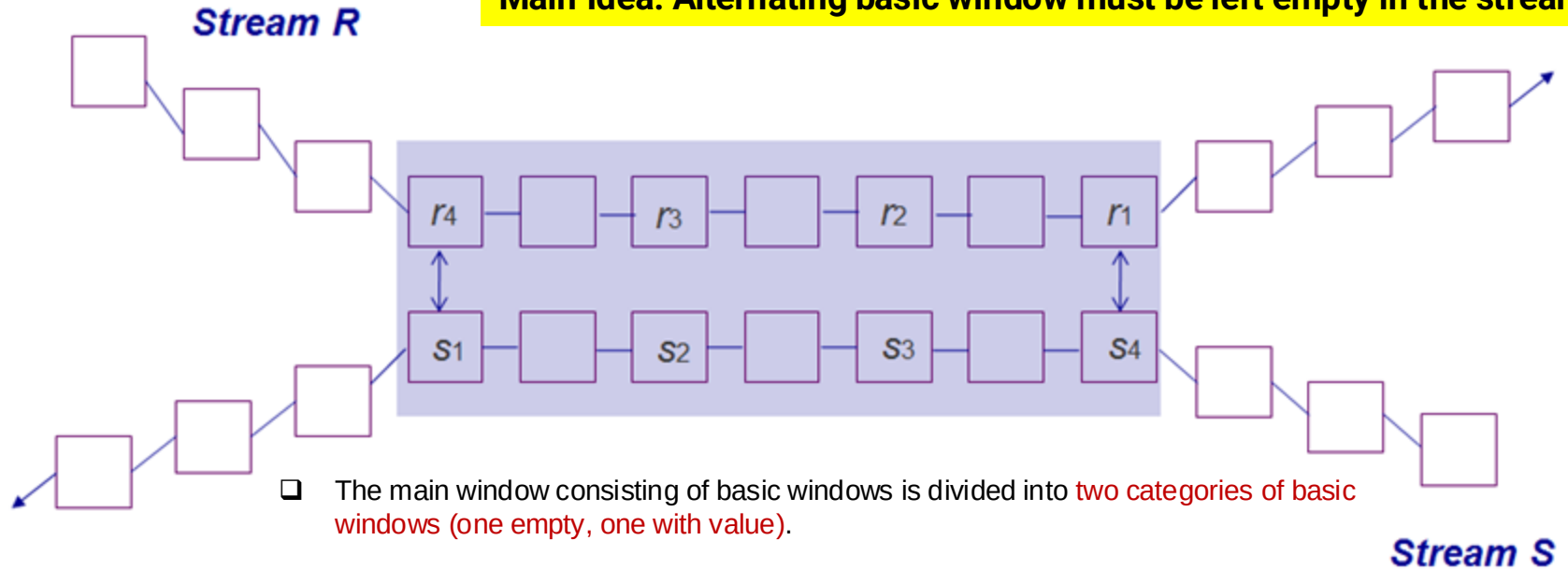
Handshake Join



Handshake Join (Solution 1)

Solution 1:

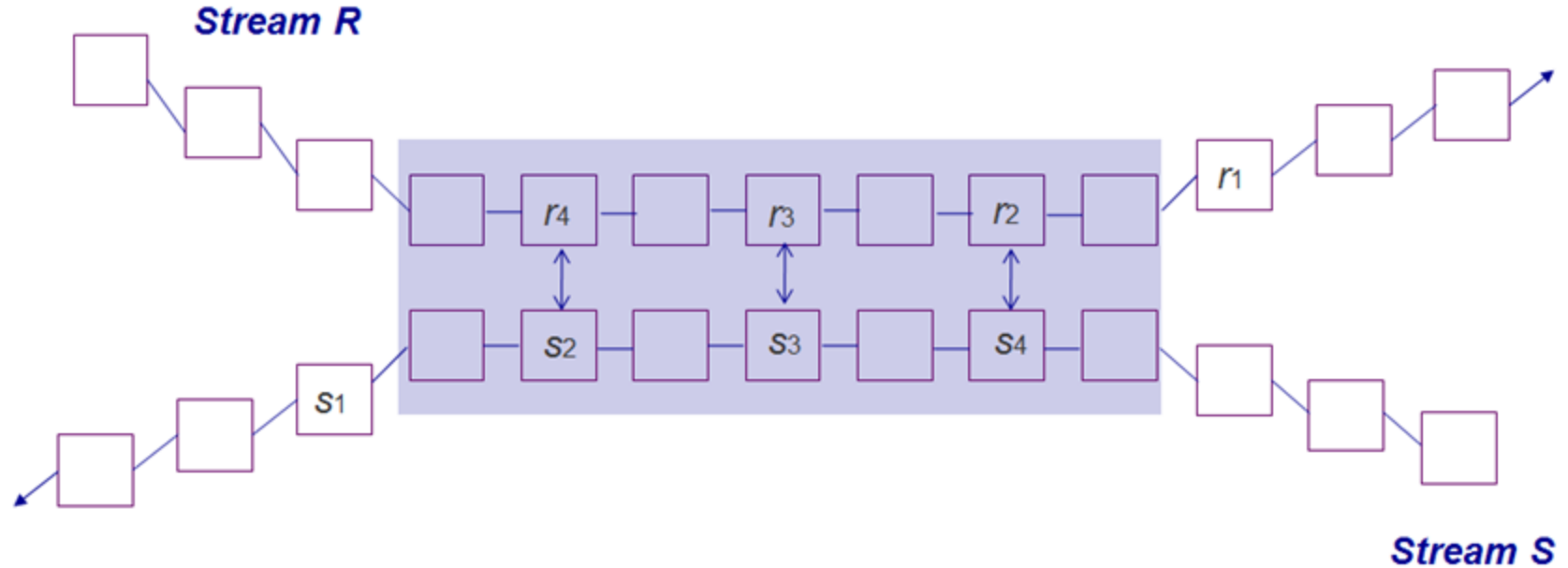
Main Idea: Alternating basic window must be left empty in the stream.



- ❑ The main window consisting of basic windows is divided into **two categories of basic windows (one empty, one with value)**.
- ❑ Each basic window will be used in an alternate fashion. When the streams move, they will move to the empty basic window, and hence no missing handshake will occur.

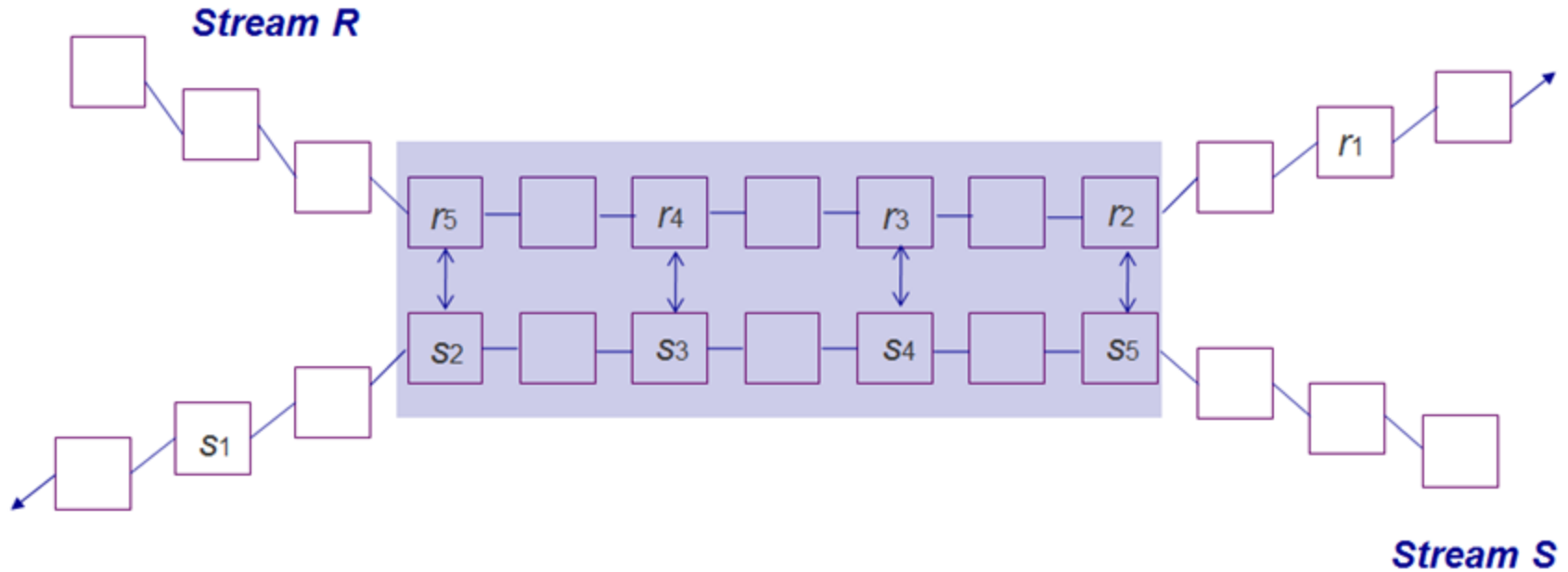
Handshake Join (Solution 1)

Solution 1:



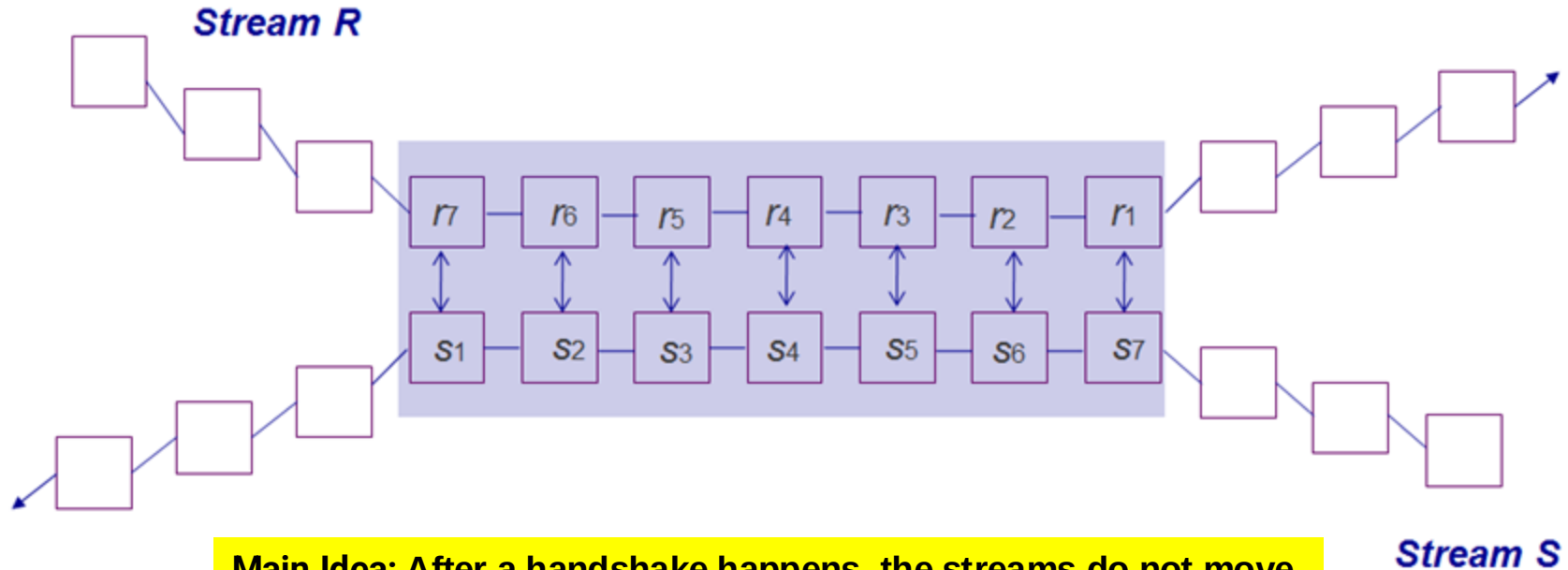
Handshake Join (Solution 1)

Solution 1:



Handshake Join (Solution 2)

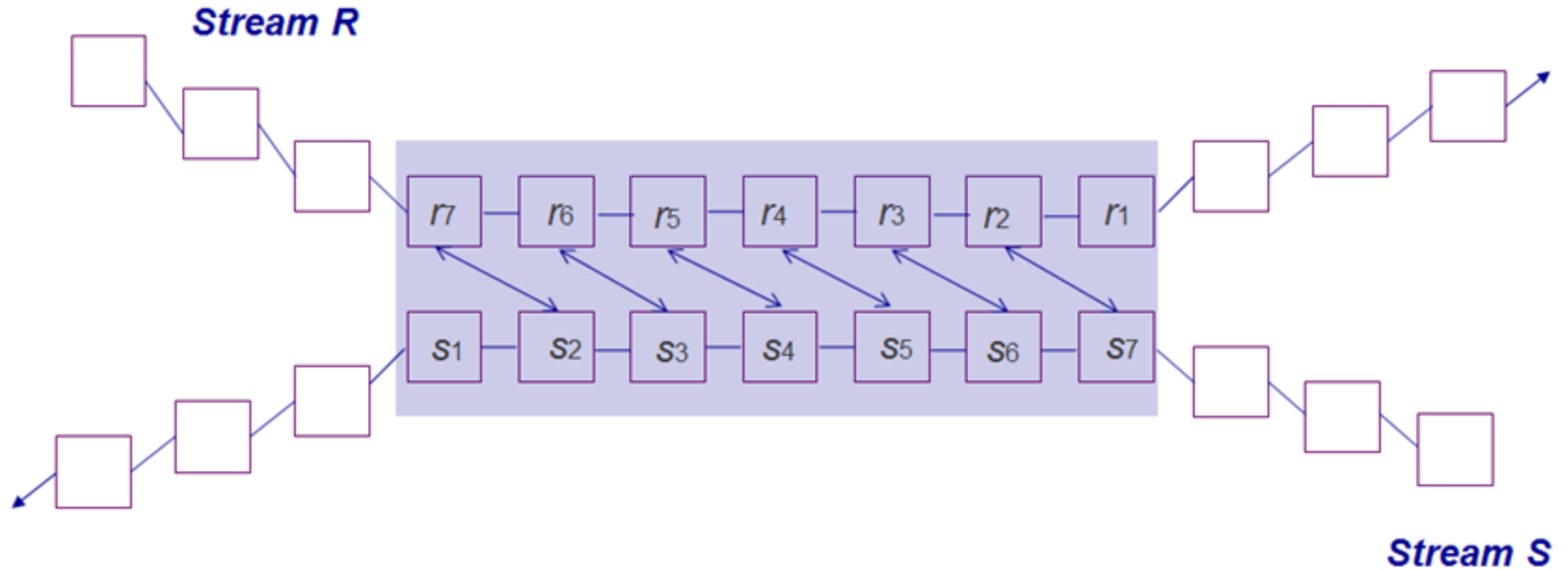
Solution 2:



Main Idea: After a handshake happens, the streams do not move forward, but each tuple in the stream performs a handshake with the next tuple, and then after that both streams move forward.

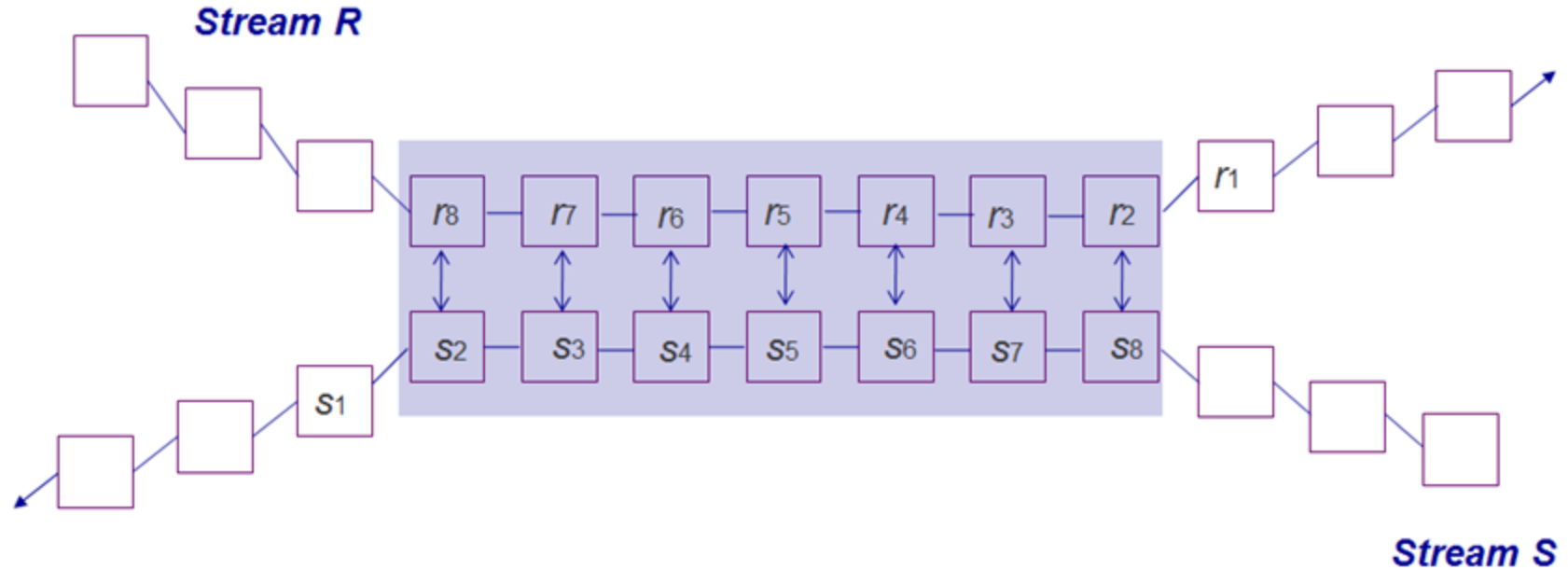
Handshake Join (Solution 2)

Solution 2:



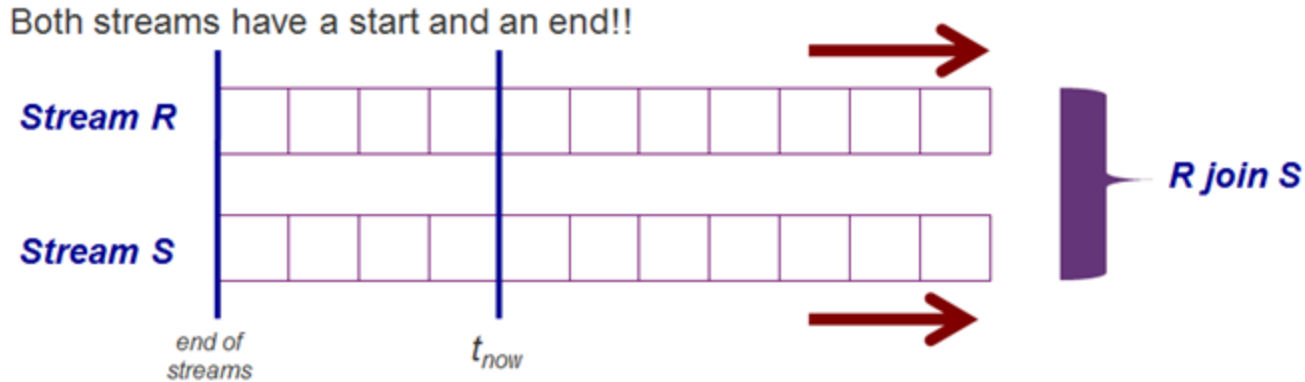
Handshake Join (Solution 2)

Solution 2:



- Overview of Stream join
- Time based window stream join (Unbounded)
 - Tuple slide
 - Time slide
- Tuple based window stream join (Unbounded)
- **Bounded stream join**
 - ❑ How to join two data streams that has end?
e.g. data from railway sensor, when train stops, data is not streaming

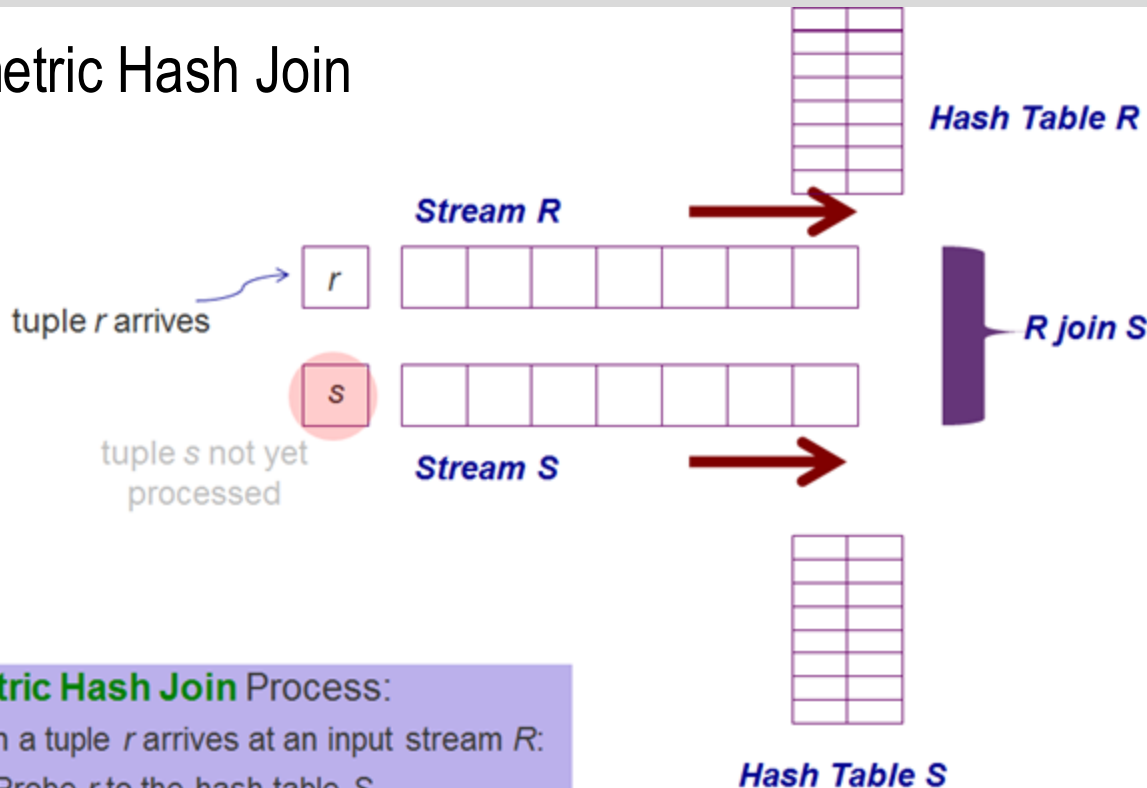
Bounded Stream Join



- No window → it's easy...
- The semantic is hence the same as that of relational join
- It is also called a "Pipelining Join"
- Processing options:
 - Offline (the same as relational join processing) → Wait until all data come in, and perform traditional relational join processing
 - Online (this is pipelining join) → Perform join while data is streaming

Symmetric Hash Join

Step 1:



How is it different from window-based join for unbounded stream?

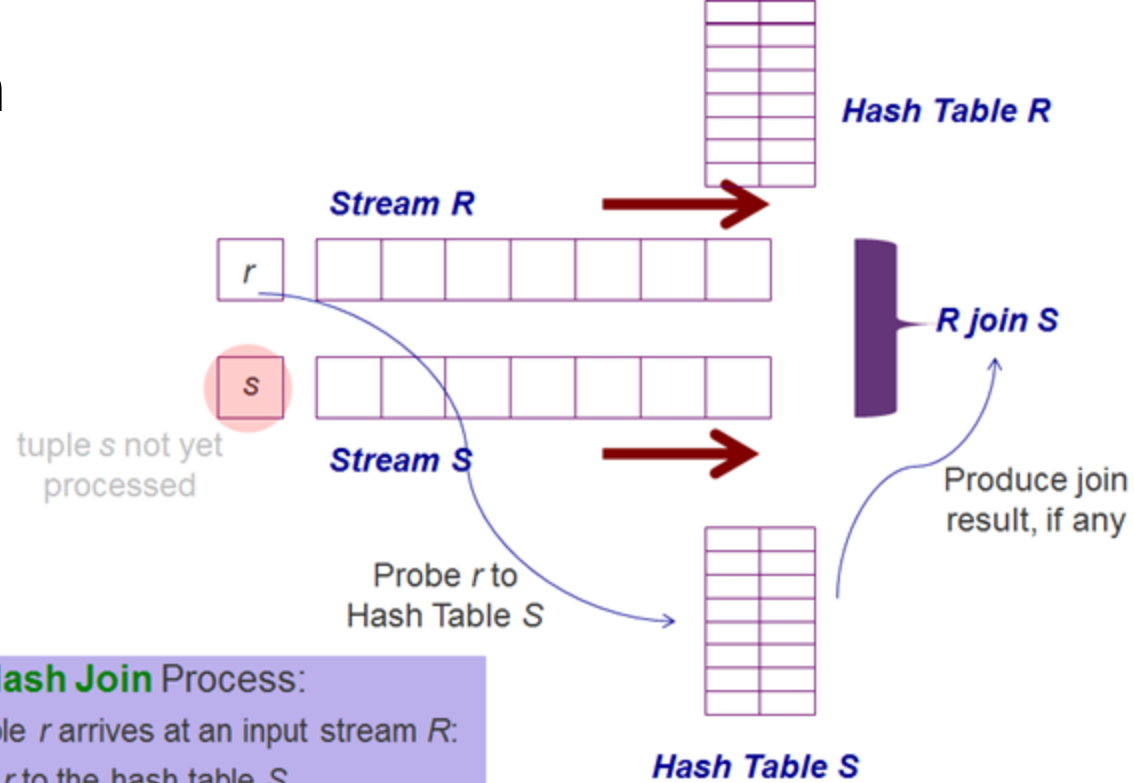
Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

- ❑ Window-based join:
Remove entries from hash table when tuple expire
- ❑ Bounded stream join:
 - No removal of entries from hash table
 - Hash tables R & S will hold entire stream data
 - because stream will end at some point

Symmetric Hash Join

Step 2:

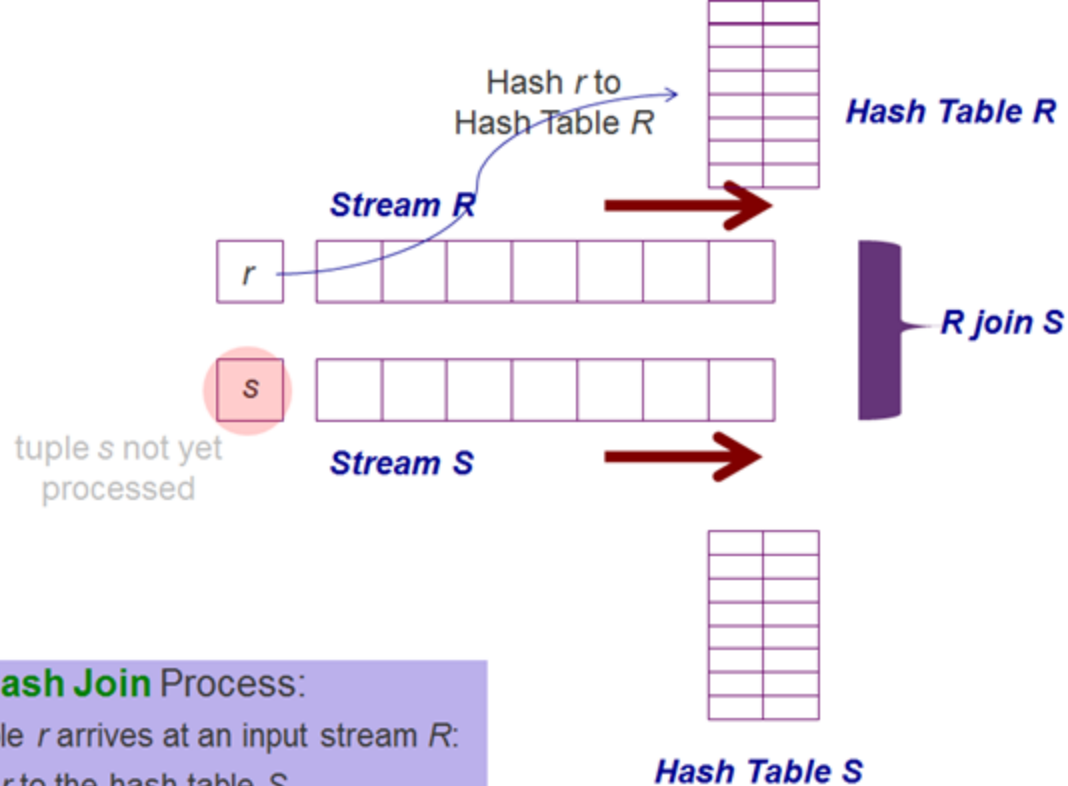


Symmetric Hash Join Process:

- When a tuple *r* arrives at an input stream *R*:
 - Probe *r* to the hash table *S*
 - Hash tuple *r* into hash table *R*
 - Insert new tuple *r* into stream *R*

Symmetric Hash Join

Step 3:

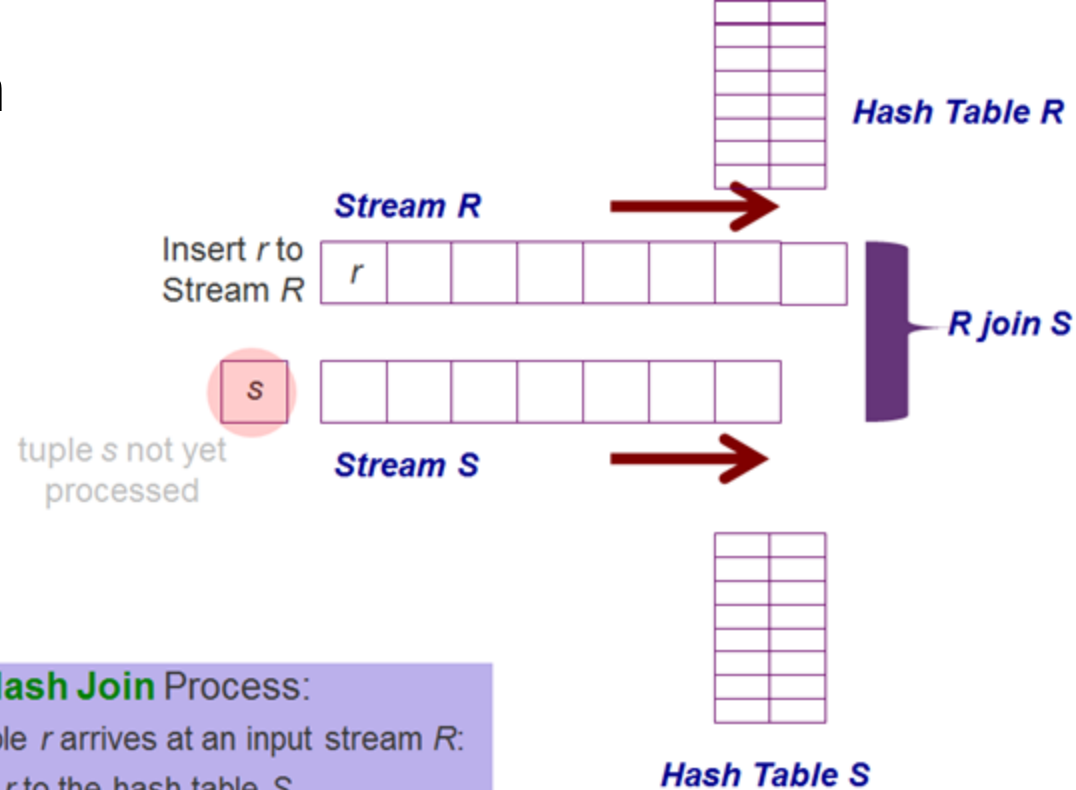


Symmetric Hash Join Process:

- When a tuple *r* arrives at an input stream *R*:
 - Probe *r* to the hash table *S*
 - Hash tuple *r* into hash table *R*
 - Insert new tuple *r* into stream *R*

Symmetric Hash Join

Step 4:

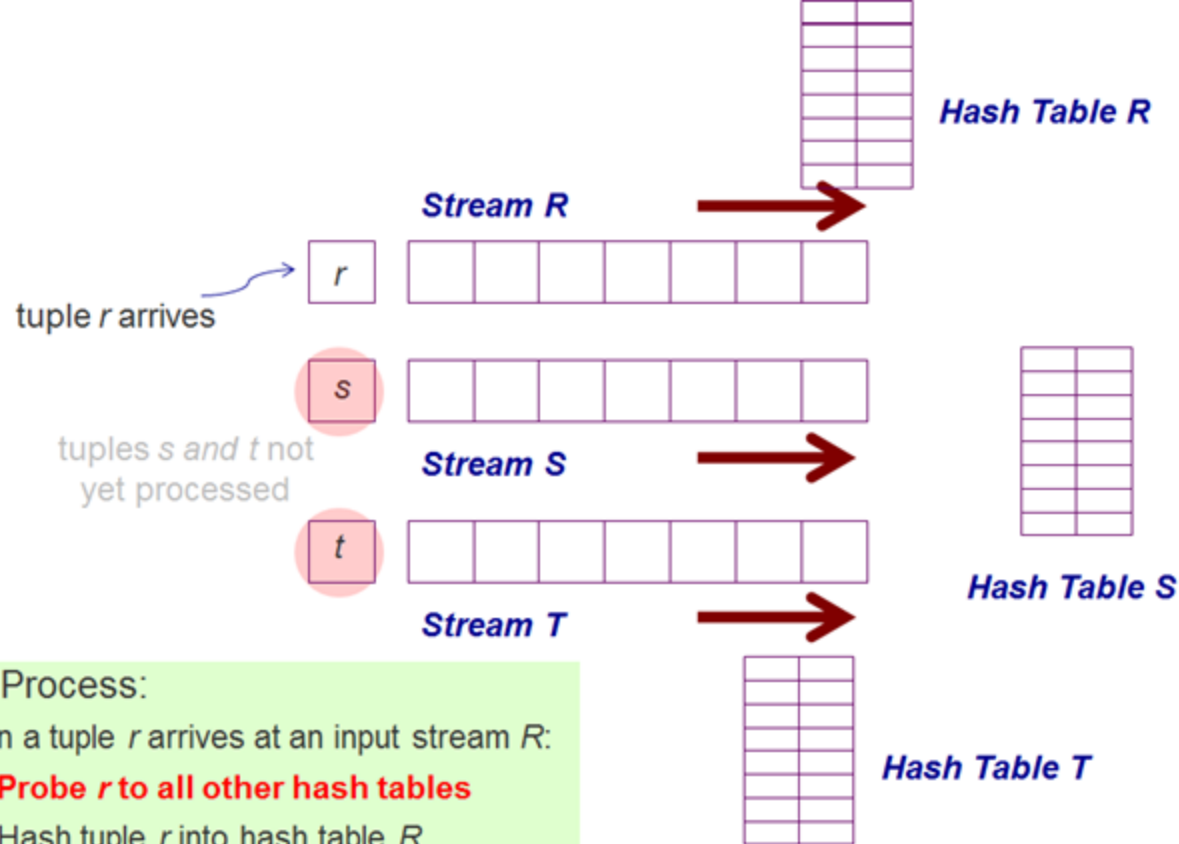


Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

M-Join

Step 1:

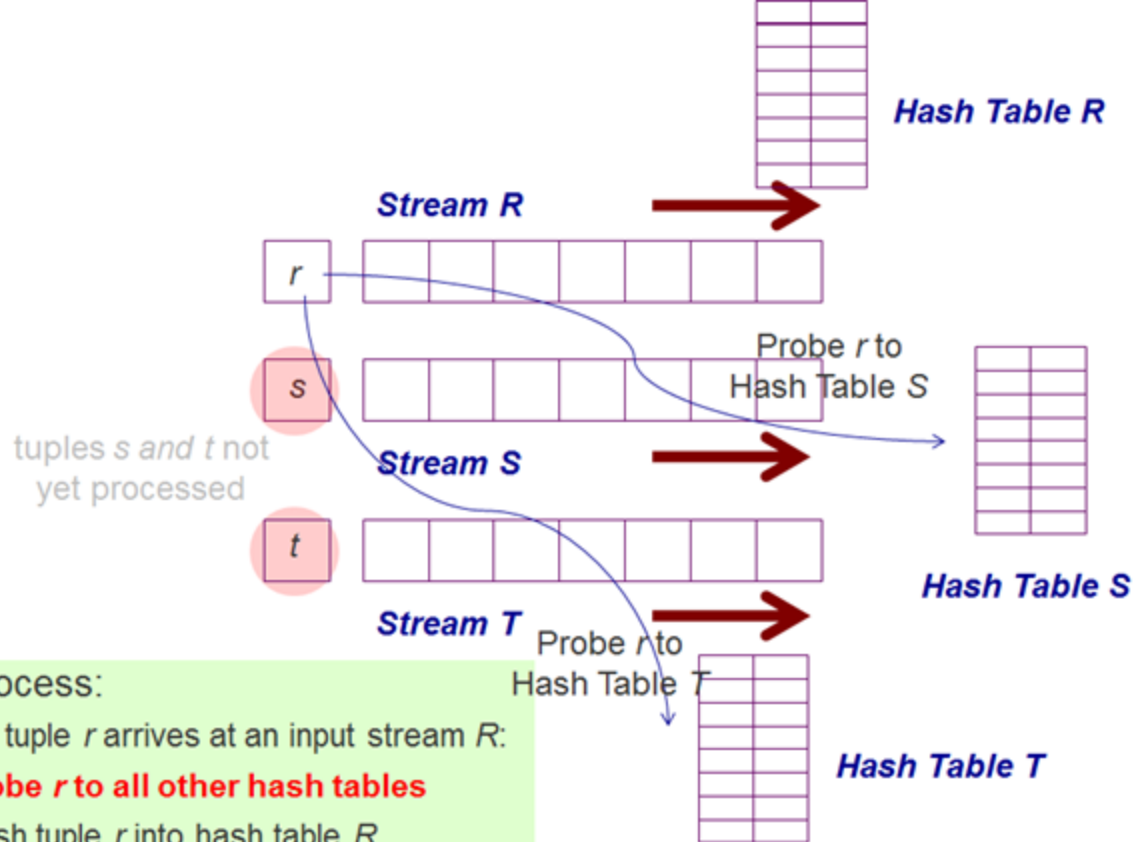


M-Join Process:

- When a tuple r arrives at an input stream R :
 - **Probe r to all other hash tables**
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

M-Join

Step 2:

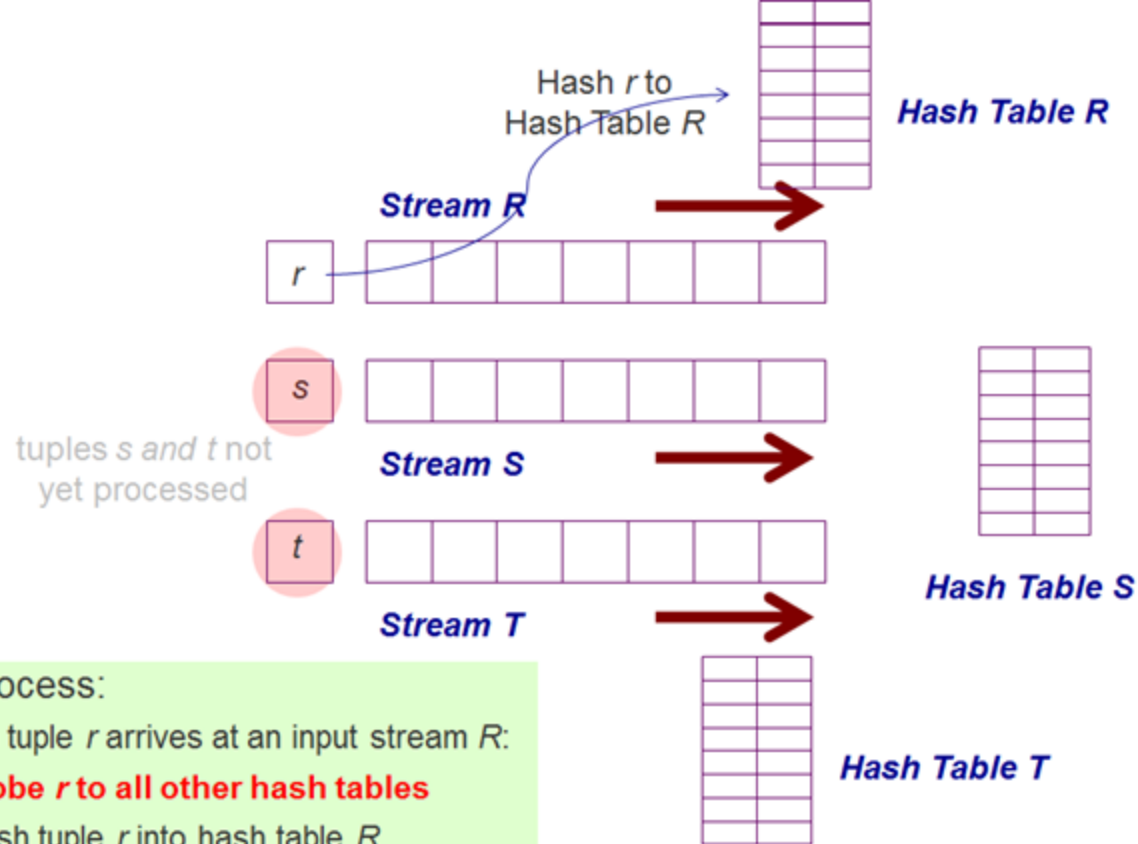


M-Join Process:

- When a tuple r arrives at an input stream R :
 - **Probe r to all other hash tables**
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

M-Join

Step 3:

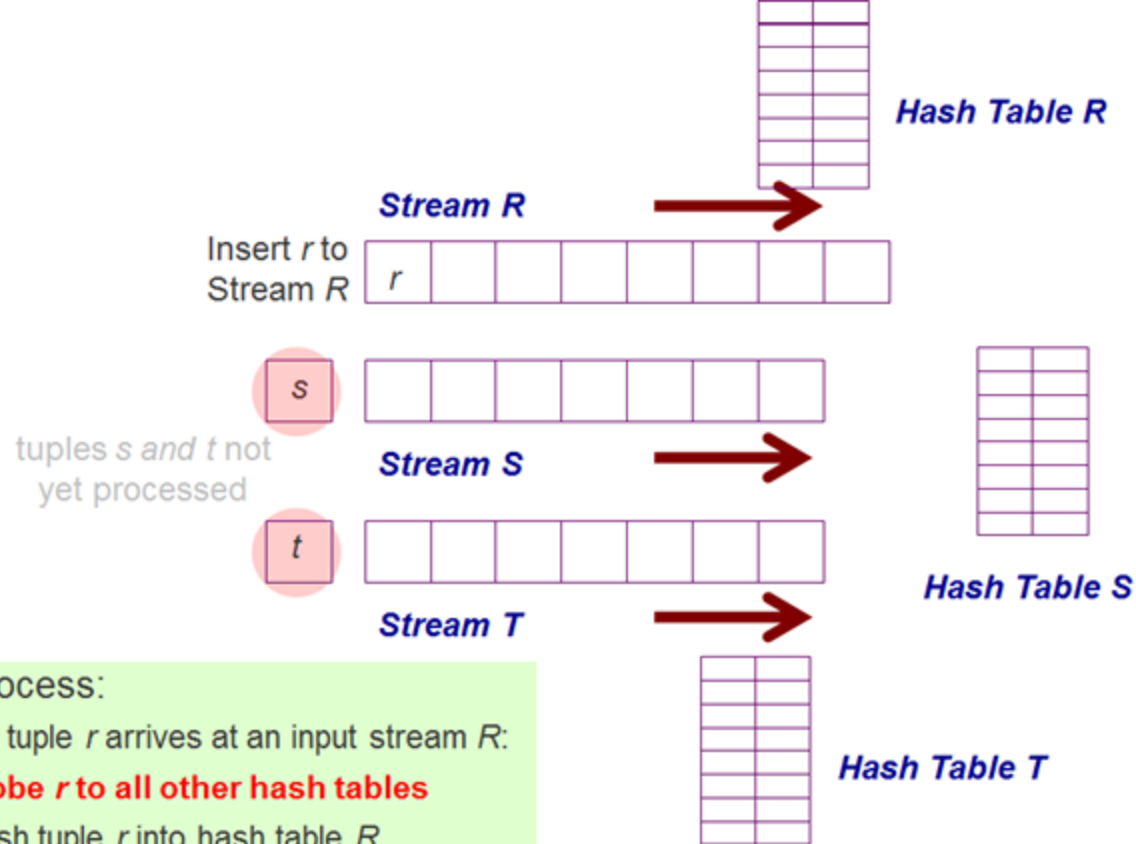


M-Join Process:

- When a tuple r arrives at an input stream R :
 - **Probe r to all other hash tables**
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

M-Join

Step 3:



M-Join Process:

- When a tuple r arrives at an input stream R :
 - **Probe r to all other hash tables**
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R